

05_Model_Development

March 30, 2026

1 Project Progress Report 2 – Model Development Phase

1.1 Port Authority of New York & New Jersey

In the previous phases of this project, we completed data auditing, exploratory analysis, cleaning, integration, and sampling in order to create a structured and analytically reliable dataset.

A representative sample was already created using a stratified sampling approach across facility and time. This ensured that the sample preserves the original distribution, seasonal patterns, and operational characteristics of the full dataset while allowing efficient model development.

At this stage, we use this validated sample dataset as the foundation for all modeling and analysis. No further sampling is performed in this phase.

The objective of this phase is to move beyond descriptive analysis and answer key business questions relevant to the Port Authority. These questions focus on identifying traffic drivers, understanding operational patterns, evaluating potential behavioral shifts, and forecasting future demand.

Since different questions require different types of analysis, the analytical approach is selected based on the nature of each problem. Where prediction of a continuous variable is required, regression techniques are applied. Where pattern identification is required, classification methods may be used. For future projections, time-based methods are considered.

To ensure that model selection is not based on assumptions, H2O AutoML is used within Python to evaluate multiple candidate models and identify the best-performing approach for each task.

In this notebook, we begin with **Project Question 1**:

What are the top five factors that affect the usage of bridges and tunnels by drivers?

```
[3]: import pandas as pd
import numpy as np
import os

import h2o
from h2o.automl import H2OAutoML
```

```
[4]: print("Current Working Directory:")
print(os.getcwd())

print("\nFiles in current folder:")
print(os.listdir())
```

```
print("\nFiles in data folder:")
print(os.listdir("../data"))
```

Current Working Directory:

C:\Users\nanda\OneDrive\Desktop\Project\PortAuthority_Capstone\notebooks

Files in current folder:

```
['.ipynb_checkpoints', '01_data_audit.ipynb',
'02_descriptive_analysis_and_eda.ipynb',
'03_data_cleaning_and_grain_alignment.ipynb', '04_Sampling.ipynb',
'Untitled.ipynb']
```

Files in data folder:

```
['cleaned', 'integrated', 'processed', 'raw']
```

```
[6]: traffic_data = pd.read_csv("../data/integrated/final_master_sample.csv")

print("Shape of dataset:", traffic_data.shape)
traffic_data.head()
```

Shape of dataset: (3471, 21)

```
[6]:
```

	FAC	DATE	TOTAL	CASH	EZPASS	VIOLATION	Autos	Small_T	Large_T	\
0	1	2013-01-28	39411	7105	31576	730	37442	1279	32	
1	1	2013-01-16	41417	6837	33738	842	39383	1362	35	
2	1	2013-01-24	44540	7570	36180	790	42416	1432	18	
3	1	2013-02-10	33051	7361	25124	566	32540	213	5	
4	1	2013-02-26	43210	7283	35111	816	41056	1488	22	

	Buses	...	AWND	PRCP	SNOW	TMAX	TMIN	TAVG	Passenger_Volume	\
0	658	...	2.1	5.6	0	2.2	-1.7	0.25	0.0	
1	637	...	2.2	17.5	0	2.8	0.0	1.40	0.0	
2	674	...	3.8	0.0	0	-5.6	-11.1	-8.35	0.0	
3	293	...	1.4	0.0	0	2.2	-7.8	-2.80	0.0	
4	644	...	3.4	3.6	0	6.7	1.7	4.20	0.0	

	Month_Year	Bus_Volume	DayOfWeek
0	2013-01-01	0.0	Monday
1	2013-01-01	0.0	Wednesday
2	2013-01-01	0.0	Thursday
3	2013-02-01	0.0	Sunday
4	2013-02-01	0.0	Tuesday

[5 rows x 21 columns]

```
[7]: print("\nColumn names:")
print(traffic_data.columns.tolist())
```

```

print("\nData types:")
print(traffic_data.dtypes)

print("\nMissing values per column:")
print(traffic_data.isnull().sum())

traffic_data.describe().T

print("TOTAL min:", traffic_data["TOTAL"].min())
print("TOTAL max:", traffic_data["TOTAL"].max())

print("\nFAC unique values:", traffic_data["FAC"].unique())
print("\nDayOfWeek unique values:", traffic_data["DayOfWeek"].unique())

```

Column names:

```

['FAC', 'DATE', 'TOTAL', 'CASH', 'EZPASS', 'VIOLATION', 'Autos', 'Small_T',
'Large_T', 'Buses', 'STATION', 'AWND', 'PRCP', 'SNOW', 'TMAX', 'TMIN', 'TAVG',
'Passenger_Volume', 'Month_Year', 'Bus_Volume', 'DayOfWeek']

```

Data types:

```

FAC                int64
DATE               object
TOTAL              int64
CASH               int64
EZPASS             int64
VIOLATION          int64
Autos              int64
Small_T            int64
Large_T            int64
Buses              int64
STATION            object
AWND               float64
PRCP               float64
SNOW               int64
TMAX               float64
TMIN               float64
TAVG               float64
Passenger_Volume   float64
Month_Year          object
Bus_Volume          float64
DayOfWeek           object
dtype: object

```

Missing values per column:

```

FAC    0
DATE    0

```

```

TOTAL          0
CASH           0
EZPASS        0
VIOLATION      0
Autos          0
Small_T        0
Large_T        0
Buses          0
STATION        0
AWND           0
PRCP           0
SNOW           0
TMAX           0
TMIN           0
TAVG           0
Passenger_Volume 0
Month_Year     0
Bus_Volume     0
DayOfWeek      0
dtype: int64
TOTAL min: 9
TOTAL max: 95214

```

```
FAC unique values: [1 2 4 5 6 7 8 9]
```

```
DayOfWeek unique values: ['Monday' 'Wednesday' 'Thursday' 'Sunday' 'Tuesday'
'Saturday' 'Friday']
```

1.1.1 Step 2 — Prepare the dataset for H2O AutoML

After loading and inspecting the sample dataset, the next step is to prepare it for modeling.

At this stage, the goal is to answer Project Question 1 by identifying the factors most strongly associated with total traffic volume. Since **TOTAL** is a continuous numeric variable, this is treated as a regression problem.

Before running H2O AutoML, the dataset must be prepared in a format suitable for modeling:

- **DATE** and **Month_Year** should be converted into date format
- **FAC** and **DayOfWeek** should be treated as categorical variables
- **STATION** will be excluded because it does not add meaningful explanatory value for this question
- the data must be converted into an H2O frame so that AutoML can test multiple regression models

The purpose of this first AutoML run is to allow H2O to compare different regression algorithms and identify the best-performing model for predicting traffic volume.

```
[8]: h2o.init()
```

```

Checking whether there is an H2O instance running at http://localhost:54321...
not found.
Attempting to start a local H2O server...
; Java HotSpot(TM) 64-Bit Server VM (build 17.0.16+12-LTS-247, mixed mode,
sharing)
Starting server from C:\Users\nanda\anaconda3\Lib\site-
packages\h2o\backend\bin\h2o.jar
Ice root: C:\Users\nanda\AppData\Local\Temp\tmp_b5as2ql
JVM stdout:
C:\Users\nanda\AppData\Local\Temp\tmp_b5as2ql\h2o_nanda_started_from_python.out
JVM stderr:
C:\Users\nanda\AppData\Local\Temp\tmp_b5as2ql\h2o_nanda_started_from_python.err
Server is running at http://127.0.0.1:54321
Connecting to H2O server at http://127.0.0.1:54321 ... successful.

```

```

-----
H2O_cluster_uptime:      02 secs
H2O_cluster_timezone:    America/New_York
H2O_data_parsing_timezone: UTC
H2O_cluster_version:     3.46.0.10
H2O_cluster_version_age: 17 days
H2O_cluster_name:        H2O_from_python_nanda_vygxxk
H2O_cluster_total_nodes: 1
H2O_cluster_free_memory: 3.951 Gb
H2O_cluster_total_cores: 0
H2O_cluster_allowed_cores: 0
H2O_cluster_status:      locked, healthy
H2O_connection_url:       http://127.0.0.1:54321
H2O_connection_proxy:     {"http": null, "https": null}
H2O_internal_security:    False
Python_version:           3.13.9 final
-----

```

```
[9]: h2o.init(nthreads=-1, max_mem_size="4G")
```

```

Checking whether there is an H2O instance running at http://localhost:54321.
connected.

```

```

-----
H2O_cluster_uptime:      2 mins 01 secs
H2O_cluster_timezone:    America/New_York
H2O_data_parsing_timezone: UTC
H2O_cluster_version:     3.46.0.10
H2O_cluster_version_age: 17 days
H2O_cluster_name:        H2O_from_python_nanda_vygxxk
H2O_cluster_total_nodes: 1
H2O_cluster_free_memory: 3.932 Gb
H2O_cluster_total_cores: 8
H2O_cluster_allowed_cores: 8

```

```

H2O_cluster_status:      locked, healthy
H2O_connection_url:      http://localhost:54321
H2O_connection_proxy:    {"http": null, "https": null}
H2O_internal_security:   False
Python_version:          3.13.9 final
-----

```

1.1.2 Step 6 — Refined predictor selection for meaningful interpretation

At this stage, it is important to ensure that the model provides meaningful business insights rather than mechanically reconstructing the target variable.

Variables such as CASH, EZPASS, VIOLATION, and vehicle category counts (Autos, Small_T, Large_T, Buses) are direct components of TOTAL traffic. Including them would lead to artificially strong model performance without providing actionable insight.

Therefore, for this analysis, these variables are excluded. The model is instead built using:

- facility information
- day-of-week patterns
- weather variables
- passenger and bus-related volumes

This ensures that the resulting model identifies real external drivers of traffic rather than simply decomposing the total.

```

[13]: question1_data = traffic_data.copy()

# Drop STATION because it is not useful for this modeling question
question1_data = question1_data.drop(columns=["STATION"])

# Convert date columns
question1_data["DATE"] = pd.to_datetime(question1_data["DATE"])
question1_data["Month_Year"] = pd.to_datetime(question1_data["Month_Year"])

print("Shape after preparation:", question1_data.shape)
question1_data.head()

```

Shape after preparation: (3471, 20)

```

[13]:
   FAC  DATE  TOTAL  CASH  EZPASS  VIOLATION  Autos  Small_T  Large_T  \
0    1  2013-01-28  39411  7105   31576         730  37442    1279     32
1    1  2013-01-16  41417  6837   33738         842  39383    1362     35
2    1  2013-01-24  44540  7570   36180         790  42416    1432     18
3    1  2013-02-10  33051  7361   25124         566  32540     213      5
4    1  2013-02-26  43210  7283   35111         816  41056    1488     22

```

	Buses	AWND	PRCP	SNOW	TMAX	TMIN	TAVG	Passenger_Volume	Month_Year	\
0	658	2.1	5.6	0	2.2	-1.7	0.25	0.0	2013-01-01	
1	637	2.2	17.5	0	2.8	0.0	1.40	0.0	2013-01-01	
2	674	3.8	0.0	0	-5.6	-11.1	-8.35	0.0	2013-01-01	
3	293	1.4	0.0	0	2.2	-7.8	-2.80	0.0	2013-02-01	
4	644	3.4	3.6	0	6.7	1.7	4.20	0.0	2013-02-01	

	Bus_Volume	DayOfWeek
0	0.0	Monday
1	0.0	Wednesday
2	0.0	Thursday
3	0.0	Sunday
4	0.0	Tuesday

```
[14]: question1_h2o = h2o.H2OFrame(question1_data)

question1_h2o["FAC"] = question1_h2o["FAC"].asfactor()
question1_h2o["DayOfWeek"] = question1_h2o["DayOfWeek"].asfactor()
```

```
Parse progress:
| (done) 100%
```

```
[15]: target_column = "TOTAL"

predictor_columns = [
    "FAC",
    "AWND",
    "PRCP",
    "SNOW",
    "TMAX",
    "TMIN",
    "TAVG",
    "Passenger_Volume",
    "Bus_Volume",
    "DayOfWeek"
]

print("Target variable:", target_column)
print("Predictors:", predictor_columns)
```

```
Target variable: TOTAL
Predictors: ['FAC', 'AWND', 'PRCP', 'SNOW', 'TMAX', 'TMIN', 'TAVG',
'Passenger_Volume', 'Bus_Volume', 'DayOfWeek']
```

```
[16]: train_q1, test_q1 = question1_h2o.split_frame(ratios=[0.8], seed=42)

print("Training rows:", train_q1.nrows)
print("Testing rows:", test_q1.nrows)
```

Training rows: 2784

Testing rows: 687

```
[17]: automl_q1 = H2OAutoML(
      max_models=20,
      seed=42,
      sort_metric="RMSE",
      nfolds=5
    )

    automl_q1.train(
      x=predictor_columns,
      y=target_column,
      training_frame=train_q1,
      leaderboard_frame=test_q1
    )
```

AutoML progress: |

22:25:10.854: AutoML: XGBoost is not available; skipping it.

| (done) 100%

[17]: Model Details

=====

H2OStackedEnsembleEstimator : Stacked Ensemble

Model Key: StackedEnsemble_AllModels_1_AutoML_1_20260329_222510

Model Summary for Stacked Ensemble:

key	value
Stacking strategy	cross_validation
Number of base models (used / total)	8/20
# GBM base models (used / total)	7/10
# DRF base models (used / total)	0/2
# DeepLearning base models (used / total)	1/7
# GLM base models (used / total)	0/1
Metalearner algorithm	GLM
Metalearner fold assignment scheme	Random
Metalearner nfolds	5
Metalearner fold_column	
Custom metalearner hyperparameters	None

ModelMetricsRegressionGLM: stackedensemble

** Reported on train data. **

MSE: 6581256.7356150625

RMSE: 2565.3960192561035

MAE: 1804.2435733023483
 RMSLE: NaN
 Mean Residual Deviance: 6581256.7356150625
 R²: 0.9817133588341134
 Null degrees of freedom: 2783
 Residual degrees of freedom: 2775
 Null deviance: 1001945550620.4205
 Residual deviance: 18322218751.952335
 AIC: 51628.715551724206

ModelMetricsRegressionGLM: stackedensemble
 ** Reported on cross-validation data. **

MSE: 19979428.121441364
 RMSE: 4469.83535730807
 MAE: 3095.7432473841573
 RMSLE: NaN
 Mean Residual Deviance: 19979428.121441364
 R²: 0.9444852788101605
 Null degrees of freedom: 2783
 Residual degrees of freedom: 2775
 Null deviance: 1002108015560.7352
 Residual deviance: 55622727890.09276
 AIC: 54720.284716580805

Cross-Validation Metrics Summary:

	mean		sd	cv_1_valid	cv_2_valid
cv_3_valid	cv_4_valid	cv_5_valid			
-----	-----	-----	-----	-----	-----
aic		10959.3	230.89	10994.1	11285.8
10969.4	10908.6	10638.7			
loglikelihood		0	0	0	0
0	0				
mae		3094.32	141.714	3285.66	3195.98
2943.27	3007.59	3039.12			
mean_residual_deviance		1.99621e+07	1.65265e+06	2.10153e+07	2.23677e+07
1.88056e+07	1.86672e+07	1.89547e+07			
mse		1.99621e+07	1.65265e+06	2.10153e+07	2.23677e+07
1.88056e+07	1.86672e+07	1.89547e+07			
null_deviance		2.00422e+11	2.67145e+09	1.98638e+11	2.01294e+11
2.02337e+11	2.03116e+11	1.96723e+11			
r2		0.944467	0.00536919	0.941052	0.936661
0.947973	0.9489	0.947751			
residual_deviance		1.1124e+10	1.07625e+09	1.17055e+10	1.27496e+10
1.05123e+10	1.0379e+10	1.02735e+10			
rmse		4464.9	182.964	4584.25	4729.45

```

4336.54      4320.56      4353.7
rmsle              0.205045      0      0.205045      nan
nan              nan              nan

```

[tips]

Use `model.explain()` to inspect the model.

--

Use `h2o.display.toggle\_user\_tips()` to switch on/off this section.

```

[18]: leaderboard_q1 = automl_q1.leaderboard
      leaderboard_q1.head(rows=20)
      best_model_q1 = automl_q1.leader
      best_model_q1

```

[18]: Model Details

=====

H2OStackedEnsembleEstimator : Stacked Ensemble

Model Key: StackedEnsemble_AllModels_1_AutoML_1_20260329_222510

Model Summary for Stacked Ensemble:

key	value
Stacking strategy	cross_validation
Number of base models (used / total)	8/20
# GBM base models (used / total)	7/10
# DRF base models (used / total)	0/2
# DeepLearning base models (used / total)	1/7
# GLM base models (used / total)	0/1
Metalearner algorithm	GLM
Metalearner fold assignment scheme	Random
Metalearner nfolds	5
Metalearner fold_column	
Custom metalearner hyperparameters	None

ModelMetricsRegressionGLM: stackedensemble

** Reported on train data. **

MSE: 6581256.7356150625

RMSE: 2565.3960192561035

MAE: 1804.2435733023483

RMSLE: NaN

Mean Residual Deviance: 6581256.7356150625

R^2: 0.9817133588341134

Null degrees of freedom: 2783

Residual degrees of freedom: 2775

Null deviance: 1001945550620.4205

Residual deviance: 18322218751.952335
AIC: 51628.715551724206

ModelMetricsRegressionGLM: stackedensemble
** Reported on cross-validation data. **

MSE: 19979428.121441364
RMSE: 4469.83535730807
MAE: 3095.7432473841573
RMSLE: NaN
Mean Residual Deviance: 19979428.121441364
R²: 0.9444852788101605
Null degrees of freedom: 2783
Residual degrees of freedom: 2775
Null deviance: 1002108015560.7352
Residual deviance: 55622727890.09276
AIC: 54720.284716580805

Cross-Validation Metrics Summary:

	mean	sd	cv_1_valid	cv_2_valid
cv_3_valid	cv_4_valid	cv_5_valid		
aic	10959.3	230.89	10994.1	11285.8
10969.4	10908.6	10638.7		
loglikelihood	0	0	0	0
0	0			
mae	3094.32	141.714	3285.66	3195.98
2943.27	3007.59	3039.12		
mean_residual_deviance	1.99621e+07	1.65265e+06	2.10153e+07	2.23677e+07
1.88056e+07	1.86672e+07	1.89547e+07		
mse	1.99621e+07	1.65265e+06	2.10153e+07	2.23677e+07
1.88056e+07	1.86672e+07	1.89547e+07		
null_deviance	2.00422e+11	2.67145e+09	1.98638e+11	2.01294e+11
2.02337e+11	2.03116e+11	1.96723e+11		
r2	0.944467	0.00536919	0.941052	0.936661
0.947973	0.9489	0.947751		
residual_deviance	1.1124e+10	1.07625e+09	1.17055e+10	1.27496e+10
1.05123e+10	1.0379e+10	1.02735e+10		
rmse	4464.9	182.964	4584.25	4729.45
4336.54	4320.56	4353.7		
rmsle	0.205045	0	0.205045	nan
nan	nan	nan		

[tips]

Use `model.explain()` to inspect the model.

--

Use ``h2o.display.toggle_user_tips()`` to switch on/off this section.

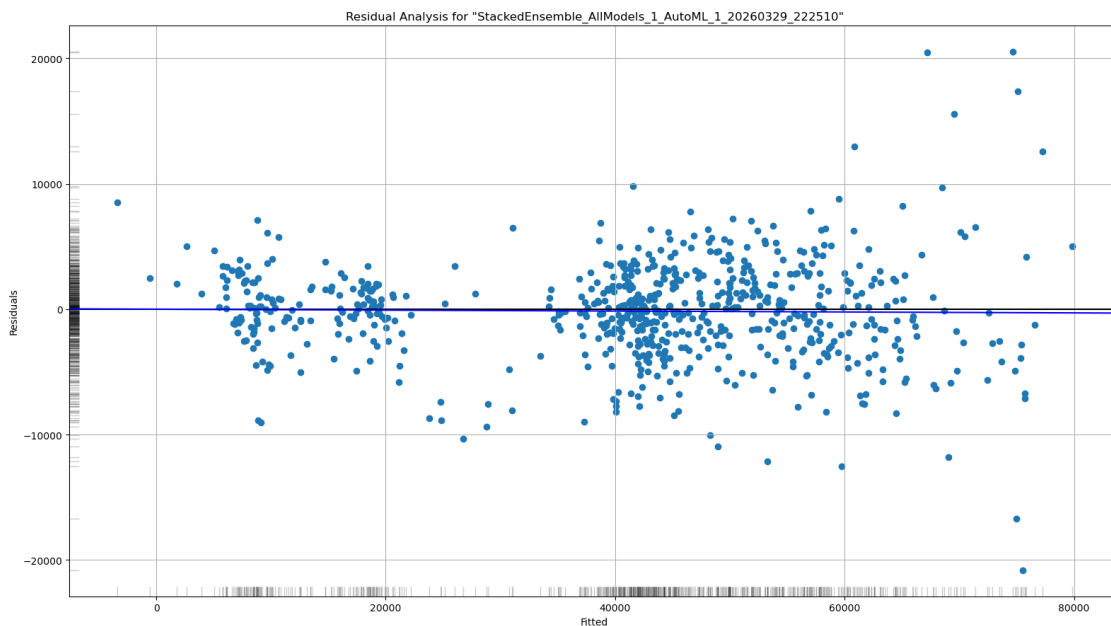
```
[19]: best_model_q1.varimp(use_pandas=True)
```

Warning: This model doesn't have variable importances

```
[20]: best_model_q1.explain(test_q1)
```

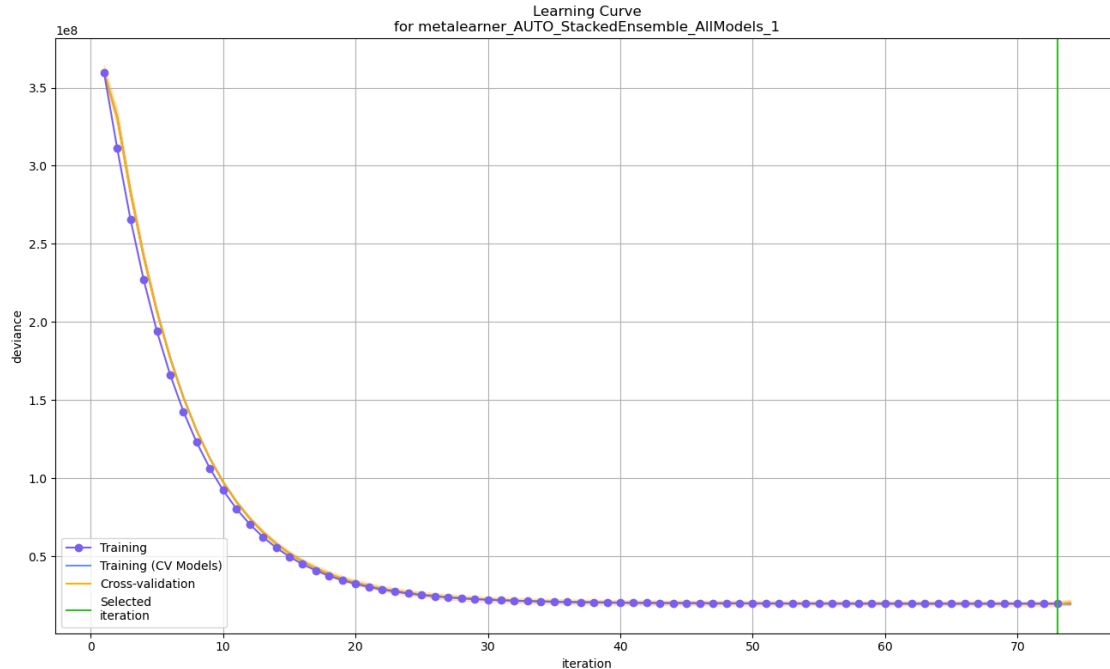
2 Residual Analysis

Residual Analysis plots the fitted values vs residuals on a test dataset. Ideally, residuals should be randomly distributed. Patterns in this plot can indicate potential problems with the model selection, e.g., using simpler model than necessary, not accounting for heteroscedasticity, autocorrelation, etc. Note that if you see “striped” lines of residuals, that is an artifact of having an integer valued (vs a real valued) response variable.



3 Learning Curve Plot

Learning curve plot shows the loss function/metric dependent on number of iterations or trees for tree-based algorithms. This plot can be useful for determining whether the model overfits.



4 Partial Dependence Plots

Partial dependence plot (PDP) gives a graphical depiction of the marginal effect of a variable on the response. The effect of a variable is measured in change in the mean response. PDP assumes independence between the feature for which is the PDP computed and the rest.

```
-----
H2OResponseError                                Traceback (most recent call last)
Cell In[20], line 1
----> 1 best_model_q1.explain(test_q1)

File ~\anaconda3\Lib\site-packages\h2o\explanation\_explain.py:3453, in
    explain(models, frame, columns, top_n_features, include_explanations,
    exclude_explanations, plot_overrides, figsize, render, qualitative_colormap,
    sequential_colormap, background_frame)
    3451 result["pdp"]["plots"][column] = H2OExplanation()
    3452 for target in targets:
-> 3453     fig = pd_plot(models_to_show[0], column=column, target=target,
    3454                  **_custom_args(plot_overrides.get(
    3455                                     ),
    3456                                     frame=frame,
    3457                                     figsize=figsize,
    3458                                     colormap=qualitative_colormap))
```

```

3458     if target is None:
3459         result["pdp"]["plots"][column] = display(fig)

File ~\anaconda3\Lib\site-packages\h2o\explanation\_explain.py:1479, in
↳ pd_plot(model, frame, column, row_index, target, max_levels, figsize,
↳ colormap, save_plot_path, binary_response_scale, grouping_column,
↳ output_graphing_data, nbins, show_rug, **kwargs)
    1411 def pd_plot(
    1412     model, # type: h2o.model.model_base.ModelBase
    1413     frame, # type: h2o.H2OFrame
    (...) 1426     **kwargs
    1427 ):
    1428     """
    1429     Plot partial dependence plot.
    1430
    (...) 1477     >>> gbm.pd_plot(test, column="alcohol")
    1478     """
-> 1479     return
↳ pd_ice_common(model, frame, column, row_index, target, max_levels, figsize, colormap, save
    1480
↳     True, binary_response_scale, None, False, grouping_column, output
    1481     show_rug=show_rug, **kwargs)

File ~\anaconda3\Lib\site-packages\h2o\explanation\_explain.py:1386, in
↳ pd_ice_common(model, frame, column, row_index, target, max_levels, figsize,
↳ colormap, save_plot_path, show_pdp, binary_response_scale, centered, is_ice,
↳ grouping_column, output_graphing_data, nbins, show_rug, **kwargs)
    1384     # decrease the number of levels to the actual number of levels
↳ in the subset
    1385     frame[column] = frame[column].ascharacter().asfactor()
-> 1386     factor_map = _factor_mapper(NumpyFrame(frame[column])).
↳ from_factor_to_num(column))
    1388 is_binomial = _is_binomial(model)
    1389 if (not is_binomial) and (binary_response_scale == "logodds"):

File ~\anaconda3\Lib\site-packages\h2o\explanation\_explain.py:236, in
↳ NumpyFrame.__init__(self, h2o_frame)
    234 _is_numeric = h2o_frame.isnumeric()
    235 self._columns = h2o_frame.columns
--> 236 self._factors = {col: h2o_frame[col].asfactor().levels()[0] for col in
    237     np.array(h2o_frame.columns)[_is_factor]}
    239 df = h2o_frame.as_data_frame(False)
    240 self._data = np.empty((h2o_frame.nrow, h2o_frame.ncol))

File ~\anaconda3\Lib\site-packages\h2o\frame.py:1438, in H2OFrame.levels(self)
    1423 def levels(self):
    1424     """
    1425     Get the factor levels.
    1426

```

```

(...) 1436     >>> h2oframe.levels()
1437     """
-> 1438     lol = _
    ↪ H2OFrame._expr(expr=ExprNode(          , self)).as_data_frame(False)
1439     lol.pop(0) # Remove column headers
1440     lol = list(zip(*lol))

File ~\anaconda3\Lib\site-packages\h2o\frame.py:1989, in H2OFrame.
    ↪ as_data_frame(self, use_pandas, header, use_multi_thread)
1986     return pandas.read_csv(StringIO(self.get_frame_data()),
    ↪ low_memory=False, skip_blank_lines=False)
1988 from h2o.utils.csv.readers import reader
-> 1989 frame = [row for row in reader(StringIO(self.get_frame_data()))]
1990 if not header:
1991     frame.pop(0)

File ~\anaconda3\Lib\site-packages\h2o\frame.py:2049, in H2OFrame.
    ↪ get_frame_data(self)
2033 def get_frame_data(self):
2034     """
2035     Get frame data as a string in csv format.
2036
(...) 2045     >>> iris.get_frame_data()
2046     """
2047     return h2o.api(
2048         "GET /3/DownloadDataset",
-> 2049         data={"frame_id": self.frame_id, "hex_string": False,
    ↪ "escape_quotes": True}
2050     )

File ~\anaconda3\Lib\site-packages\h2o\frame.py:414, in H2OFrame.frame_id(self)
402 @property
403 def frame_id(self):
404     """
405     Internal id of the frame (str).
406
(...) 412     >>> print(iris.frame_id)
413     """
--> 414     return self._frame()._ex._cache._id

File ~\anaconda3\Lib\site-packages\h2o\frame.py:584, in H2OFrame._frame(self,
    ↪ rows, rows_offset, cols, cols_offset, fill_cache)
    ↪ fill_cache=False):
--> 584     self._ex._eager_frame()
585     if fill_cache:
586         self._ex._cache.fill(rows=rows, rows_offset=rows_offset,
    ↪ cols=cols, cols_offset=cols_offset)

```

```

File ~\anaconda3\Lib\site-packages\h2o\expr.py:90, in ExprNode._eager_frame(sel.)
    88 if not self._cache.is_empty(): return
    89 if self._cache._id is not None: return # Data already computed under
↳ ID, but not cached locally
--> 90 self._eval_driver( )

File ~\anaconda3\Lib\site-packages\h2o\expr.py:114, in ExprNode.
↳ _eval_driver(self, top)
    107 """
    108 :param top: if this is a top expression (providing a final result),
    109     then specifies the expected result type (accepted values = ['frame'
↳ 'scalar']),
    110     or None if no object creation is expected.
    111 :return: self expr
    112 """
    113 exec_str = self._get_ast_str(top)
--> 114 res = ExprNode.rapids(exec_str)
    115 if 'scalar' in res:
    116     if isinstance(res['scalar'], list):

File ~\anaconda3\Lib\site-packages\h2o\expr.py:258, in ExprNode.rapids(expr)
    249 @staticmethod
    250 def rapids(expr):
    251     """
    252     Execute a Rapids expression.
    253
    (...) 256     :returns: The JSON response (as a python dictionary) of the
↳ Rapids execution
    257     """
--> 258     return
↳ h2o.api( , data={ : expr, : h2o.connection().session_id})

File ~\anaconda3\Lib\site-packages\h2o\h2o.py:123, in api(endpoint, data, json,
↳ filename, save_to)
    121 # type checks are performed in H2OConnection class
    122 _check_connection()
--> 123 return
↳ h2oconn.request(endpoint, data=data, json=json, filename=filename, save_to=save_to)

File ~\anaconda3\Lib\site-packages\h2o\backend\connection.py:499, in
↳ H2OConnection.request(self, endpoint, data, json, filename, save_to)
    497     save_to = save_to(resp)
    498     self._log_end_transaction(start_time, resp)
--> 499     return self._process_response(resp, save_to)
    501 except (requests.exceptions.ConnectionError, requests.exceptions.
↳ HTTPError) as e:
    502     if self._local_server and not self._local_server.is_running():

```

```

File ~\anaconda3\Lib\site-packages\h2o\backend\connection.py:853, in
↳H2OConnection._process_response(response, save_to)
    851 if status_code in {400, 404, 412} and isinstance(data, H2OErrorV3):
    852     data.show_stacktrace = False
--> 853     raise H2OResponseError(data)
    855 # Server errors (notably 500 = "Server Error")
    856 # Note that it is possible to receive valid H2OErrorV3 object in this
↳case, however it merely means the server
    857 # did not provide the correct status code.
    858 raise H2OServerError("HTTP %d %s:\n%s" % (status_code, response.reason,
↳data))

```

```

H2OResponseError: Server error java.lang.IllegalArgumentException:
Error: Incorrect number of arguments; 'cols_py' expects 2 but was passed 3
Request: POST /99/Rapids
data: {'ast': "(tmp= py_18_sid_b3f0 (levels (tmp= py_17_sid_b3f0 (as.factor
↳(cols_py py_16_sid_b3f0 np.str_('FAC'))))))", 'session_id': '_sid_b3f0'}

```

```

[22]: leaderboard_q1_df = automl_q1.leaderboard.as_data_frame()
leaderboard_q1_df

```

```

C:\Users\nanda\anaconda3\Lib\site-packages\h2o\frame.py:1983:
H2ODependencyWarning: Converting H2O frame to pandas dataframe using single-
thread. For faster conversion using multi-thread, install polars and pyarrow
and use it as pandas_df = h2o_df.as_data_frame(use_multi_thread=True)

```

```

warnings.warn("Converting H2O frame to pandas dataframe using single-thread.
For faster conversion using")

```

```

[22]:

```

	model_id	rmse \
0	StackedEnsemble_AllModels_1_AutoML_1_20260329_...	3948.158960
1	GBM_2_AutoML_1_20260329_222510	4001.161110
2	StackedEnsemble_BestOfFamily_1_AutoML_1_202603...	4014.918292
3	GBM_5_AutoML_1_20260329_222510	4053.376492
4	GBM_grid_1_AutoML_1_20260329_222510_model_1	4093.323535
5	GBM_4_AutoML_1_20260329_222510	4183.378032
6	GBM_grid_1_AutoML_1_20260329_222510_model_4	4226.393983
7	GBM_3_AutoML_1_20260329_222510	4252.034579
8	GBM_grid_1_AutoML_1_20260329_222510_model_5	4260.498231
9	GBM_1_AutoML_1_20260329_222510	4410.540260
10	GBM_grid_1_AutoML_1_20260329_222510_model_3	4424.500635
11	GBM_grid_1_AutoML_1_20260329_222510_model_2	4445.110068
12	DRF_1_AutoML_1_20260329_222510	4989.442074
13	DeepLearning_grid_1_AutoML_1_20260329_222510_m...	5829.023087
14	DeepLearning_grid_2_AutoML_1_20260329_222510_m...	5873.430463

```

15 DeepLearning_grid_1_AutoML_1_20260329_222510_m... 6228.795681
16 DeepLearning_grid_3_AutoML_1_20260329_222510_m... 6357.498381
17 DeepLearning_grid_3_AutoML_1_20260329_222510_m... 6472.991148
18 DeepLearning_grid_2_AutoML_1_20260329_222510_m... 7568.388263
19         DeepLearning_1_AutoML_1_20260329_222510 8222.586487
20             XRT_1_AutoML_1_20260329_222510 9176.017118
21             GLM_1_AutoML_1_20260329_222510 18426.265452

```

	mse	mae	rmse	mean_residual_deviance
0	1.558796e+07	2830.269561	NaN	1.558796e+07
1	1.600929e+07	2823.284114	NaN	1.600929e+07
2	1.611957e+07	2845.317171	NaN	1.611957e+07
3	1.642986e+07	2891.085534	NaN	1.642986e+07
4	1.675530e+07	2950.643940	NaN	1.675530e+07
5	1.750065e+07	2937.363799	NaN	1.750065e+07
6	1.786241e+07	3090.559902	0.342110	1.786241e+07
7	1.807980e+07	2933.346070	0.329008	1.807980e+07
8	1.815185e+07	3025.922153	0.318750	1.815185e+07
9	1.945287e+07	3112.105368	NaN	1.945287e+07
10	1.957621e+07	3233.124693	NaN	1.957621e+07
11	1.975900e+07	3142.514938	NaN	1.975900e+07
12	2.489453e+07	3607.403066	0.339055	2.489453e+07
13	3.397751e+07	4205.532886	NaN	3.397751e+07
14	3.449719e+07	4007.646883	NaN	3.449719e+07
15	3.879790e+07	4486.702019	0.420320	3.879790e+07
16	4.041779e+07	4462.306567	0.329056	4.041779e+07
17	4.189961e+07	4483.582774	0.333326	4.189961e+07
18	5.728050e+07	5251.078215	0.353684	5.728050e+07
19	6.761093e+07	6325.850547	0.438231	6.761093e+07
20	8.419929e+07	7148.761234	0.487382	8.419929e+07
21	3.395273e+08	14454.334027	0.775351	3.395273e+08

```

[23]: non_ensemble_models = leaderboard_q1_df[
        ~leaderboard_q1_df["model_id"].str.contains("StackedEnsemble", case=False,
        ↪na=False)
    ]

non_ensemble_models.head(10)

```

```

[23]:

```

	model_id	rmse	mse \
1	GBM_2_AutoML_1_20260329_222510	4001.161110	1.600929e+07
3	GBM_5_AutoML_1_20260329_222510	4053.376492	1.642986e+07
4	GBM_grid_1_AutoML_1_20260329_222510_model_1	4093.323535	1.675530e+07
5	GBM_4_AutoML_1_20260329_222510	4183.378032	1.750065e+07
6	GBM_grid_1_AutoML_1_20260329_222510_model_4	4226.393983	1.786241e+07
7	GBM_3_AutoML_1_20260329_222510	4252.034579	1.807980e+07
8	GBM_grid_1_AutoML_1_20260329_222510_model_5	4260.498231	1.815185e+07

```

9          GBM_1_AutoML_1_20260329_222510  4410.540260  1.945287e+07
10 GBM_grid_1_AutoML_1_20260329_222510_model_3  4424.500635  1.957621e+07
11 GBM_grid_1_AutoML_1_20260329_222510_model_2  4445.110068  1.975900e+07

```

	mae	rmsle	mean_residual_deviance
1	2823.284114	NaN	1.600929e+07
3	2891.085534	NaN	1.642986e+07
4	2950.643940	NaN	1.675530e+07
5	2937.363799	NaN	1.750065e+07
6	3090.559902	0.342110	1.786241e+07
7	2933.346070	0.329008	1.807980e+07
8	3025.922153	0.318750	1.815185e+07
9	3112.105368	NaN	1.945287e+07
10	3233.124693	NaN	1.957621e+07
11	3142.514938	NaN	1.975900e+07

```

[24]: best_interpretable_model_id = non_ensemble_models.iloc[0]["model_id"]
best_interpretable_model = h2o.get_model(best_interpretable_model_id)

print("Best interpretable model ID:", best_interpretable_model_id)
best_interpretable_model

```

Best interpretable model ID: GBM_2_AutoML_1_20260329_222510

[24]: Model Details

=====

H2OGradientBoostingEstimator : Gradient Boosting Machine

Model Key: GBM_2_AutoML_1_20260329_222510

Model Summary:

	number_of_trees	number_of_internal_trees	model_size_in_bytes	
min_depth	max_depth	mean_depth	min_leaves	max_leaves
7	121	121	78084	7
		18	46.6281	

ModelMetricsRegression: gbm

** Reported on train data. **

MSE: 8191978.121110088

RMSE: 2862.1631891124043

MAE: 1975.1585961429553

RMSLE: NaN

Mean Residual Deviance: 8191978.121110088

ModelMetricsRegression: gbm

** Reported on cross-validation data. **

MSE: 21462551.98011601

RMSE: 4632.76936401069

MAE: 3185.304543654397

RMSLE: NaN

Mean Residual Deviance: 21462551.98011601

Cross-Validation Metrics Summary:

		mean	sd	cv_1_valid	cv_2_valid
cv_3_valid	cv_4_valid	cv_5_valid			
-----	-----	-----	-----	-----	-----
aic		nan	0	nan	nan
nan	nan	nan			
loglikelihood		nan	0	nan	nan
nan	nan	nan			
mae		3184.21	90.1676	3168.82	3211.87
3242.87	3035.69	3261.8			
mean_residual_deviance		2.14801e+07	1.51049e+06	2.19025e+07	2.19751e+07
2.28253e+07	1.88768e+07	2.18209e+07			
mse		2.14801e+07	1.51049e+06	2.19025e+07	2.19751e+07
2.28253e+07	1.88768e+07	2.18209e+07			
r2		0.940324	0.00398642	0.939157	0.938747
0.93669	0.94713	0.939895			
residual_deviance		2.14801e+07	1.51049e+06	2.19025e+07	2.19751e+07
2.28253e+07	1.88768e+07	2.18209e+07			
rmse		4632.28	166.334	4680.01	4687.76
4777.59	4344.74	4671.28			
rmsle		0.263153	0	nan	0.263153
nan	nan	nan			

Scoring History:

timestamp	duration	number_of_trees	training_rmse
training_mae	training_deviance		
-----	-----	-----	-----
2026-03-29 22:25:18	0.797 sec	0.0	18970.878607398372
14978.089154002062	359894235.13664526		
2026-03-29 22:25:18	0.816 sec	5.0	13005.272282641788
10378.980451211162	169137107.14565074		
2026-03-29 22:25:18	0.833 sec	10.0	9318.286818791565
7443.987819627784	86830469.2372646		
2026-03-29 22:25:18	0.847 sec	15.0	7081.278385458762
5556.52389982377	50144503.57236546		
2026-03-29 22:25:18	0.859 sec	20.0	5648.05143383251

```

4209.74643593273    31900484.999217473
      2026-03-29 22:25:18  0.875 sec  25.0          5200.689792140148
3791.8185879060593  27047174.314070735
      2026-03-29 22:25:18  0.890 sec  30.0          4716.028966966744
3292.3924626317516  22240929.21726941
      2026-03-29 22:25:18  0.904 sec  35.0          4453.92122014701
3034.7483280225733  19837414.235275827
      2026-03-29 22:25:18  0.919 sec  40.0          4190.770092016462
2808.1955416887654  17562553.964139663
      2026-03-29 22:25:18  0.934 sec  45.0          4064.372949437434
2681.230289108452   16519127.472118745
---  ---          ---  ---          ---          ---
---
      2026-03-29 22:25:18  1.025 sec  80.0          3383.98783321021
2268.2739705162485  11451373.65531473
      2026-03-29 22:25:18  1.036 sec  85.0          3298.547009203493
2224.3421470488624  10880412.371925307
      2026-03-29 22:25:18  1.047 sec  90.0          3217.003736825211
2186.884919758501   10349113.042747373
      2026-03-29 22:25:18  1.057 sec  95.0          3144.945681814639
2148.7363745035796  9890683.341564545
      2026-03-29 22:25:18  1.073 sec  100.0         3101.7505972208364
2121.174164983048   9620856.767359816
      2026-03-29 22:25:18  1.087 sec  105.0         3022.211404765013
2076.790773325953   9133761.775091713
      2026-03-29 22:25:18  1.099 sec  110.0         2965.925042880486
2041.9837475864367   8796711.359985614
      2026-03-29 22:25:18  1.109 sec  115.0         2923.5900371575003
2011.7583371524154   8547378.705366593
      2026-03-29 22:25:18  1.120 sec  120.0         2873.2162606312872
1983.0318992548976   8255371.680356037
      2026-03-29 22:25:18  1.124 sec  121.0         2862.1631891124043
1975.1585961429553   8191978.121110088
[26 rows x 7 columns]

```

Variable Importances:

variable	relative_importance	scaled_importance	percentage
FAC	4.30426e+12	1	0.888064
DayOfWeek	1.10952e+11	0.0257772	0.0228918
TMAX	9.01908e+10	0.0209538	0.0186084
TAVG	7.75071e+10	0.0180071	0.0159914
AWND	6.29841e+10	0.014633	0.012995
TMIN	5.51183e+10	0.0128055	0.0113721
SNOW	4.40026e+10	0.010223	0.00907872
PRCP	3.78513e+10	0.00879392	0.00780956

Bus_Volume	3.58751e+10	0.00833478	0.00740182
Passenger_Volume	2.80484e+10	0.00651643	0.00578701

```
[tips]
Use `model.explain()` to inspect the model.
--
Use `h2o.display.toggle_user_tips()` to switch on/off this section.
```

```
[25]: interpretable_performance_q1 = best_interpretable_model.
      ↪ model_performance(test_q1)
      interpretable_performance_q1
```

```
[25]: ModelMetricsRegression: gbm
      ** Reported on test data. **
```

```
MSE: 16009290.231976874
RMSE: 4001.161110474917
MAE: 2823.284114011927
RMSLE: NaN
Mean Residual Deviance: 16009290.231976874
```

```
[26]: variable_importance_q1 = best_interpretable_model.varimp(use_pandas=True)
      variable_importance_q1.head(10)
```

```
[26]:
```

	variable	relative_importance	scaled_importance	percentage
0	FAC	4.304260e+12	1.000000	0.888064
1	DayOfWeek	1.109517e+11	0.025777	0.022892
2	TMAX	9.019077e+10	0.020954	0.018608
3	TAVG	7.750715e+10	0.018007	0.015991
4	AWND	6.298414e+10	0.014633	0.012995
5	TMIN	5.511828e+10	0.012806	0.011372
6	SNOW	4.400265e+10	0.010223	0.009079
7	PRCP	3.785132e+10	0.008794	0.007810
8	Bus_Volume	3.587507e+10	0.008335	0.007402
9	Passenger_Volume	2.804841e+10	0.006516	0.005787

4.0.1 Step 11 — Identifying the key drivers of traffic (Final Answer)

To answer the question “What are the top factors affecting traffic?”, we analyzed the variable importance from the best-performing interpretable model (GBM) identified by H2O AutoML.

While the stacked ensemble model provided the highest predictive accuracy, it is less suitable for direct interpretation. Therefore, the highest-ranked non-ensemble model was used to extract variable importance and identify key drivers.

The analysis revealed the following top five factors influencing traffic volume:

1. Facility (FAC)

2. Day of Week
3. Maximum Temperature (TMAX)
4. Average Temperature (TAVG)
5. Wind Speed (AWND)

The results indicate that traffic patterns are primarily driven by structural and temporal factors rather than purely demand-based variables.

Facility (FAC) emerged as the most important factor, suggesting that traffic volume varies significantly across different bridges and tunnels. This reflects differences in geographic location, connectivity, and usage patterns across Port Authority infrastructure.

Day-of-week effects further highlight predictable commuting behavior, with clear distinctions between weekday and weekend traffic patterns.

Weather variables, particularly temperature and wind, also play a meaningful role. Higher or lower temperatures influence travel demand, while wind conditions can affect driving comfort and traffic flow, especially on exposed infrastructure such as bridges.

Interestingly, passenger and bus-related variables were less influential than expected, suggesting that overall traffic volume is more strongly determined by infrastructure and environmental conditions than by passenger movement alone.

These findings provide a strong foundation for operational planning, as they identify the key external factors that Port Authority should monitor and incorporate into traffic management strategies.

4.1 Project Question 2

4.1.1 What is the question the company is asking?

The Port Authority wants to understand how many toll violations occur across its bridges and tunnels, and how these violations are distributed over time and across facilities. Specifically, the goal is to measure violation counts by year, month, week, and by facility in order to identify where and when revenue leakage is most concentrated.

4.1.2 What is the business answer to this question?

This question focuses on measuring and understanding the scale and distribution of toll violations. The objective is not to predict violations, but to quantify them and identify patterns across time and location. By doing so, the Port Authority can determine high-risk facilities and time periods where enforcement and operational interventions are most needed.

4.1.3 Models and Tools Used

Unlike Question 1, this is not a predictive modeling problem. The business requirement is to compute and analyze counts rather than forecast or classify outcomes. Therefore, regression, classification, or clustering models are not appropriate for this task.

Instead, we use descriptive analytics techniques implemented in Python, including: - Data aggregation using groupby operations - Time-based feature engineering (year, month, week) - Facility-level

breakdown analysis - Visualization for pattern identification

This approach ensures that the results are transparent, directly aligned with the business question, and easy to interpret for decision-making.

4.1.4 Why a model is NOT used here

Using a model in this case would not add value, because the company is not asking for predictions or classifications. Instead, the goal is to measure actual violation counts and understand their distribution. Applying AutoML or predictive models would introduce unnecessary complexity without improving the accuracy or usefulness of the answer.

4.1.5 What we are doing

To answer this question, we aggregate violation counts across:

- Time intervals (year, month, week)
- Facilities (bridges and tunnels)

This allows us to identify:

- overall volume of toll violations
- temporal trends (growth, seasonality)
- facility-level concentration of violations
- operational patterns relevant for enforcement

4.1.6 Why we are doing this

Understanding when and where violations occur helps the Port Authority:

- reduce revenue leakage
- prioritize enforcement resources
- identify high-risk facilities
- improve operational and policy decisions

```
[27]: # Convert DATE to datetime
      traffic_data["DATE"] = pd.to_datetime(traffic_data["DATE"])

      # Create additional time features
      traffic_data["Year"] = traffic_data["DATE"].dt.year
      traffic_data["Month"] = traffic_data["DATE"].dt.month
      traffic_data["Week"] = traffic_data["DATE"].dt.isocalendar().week
```

```
[28]: total_violations = traffic_data["VIOLATION"].sum()
      print("Total Violations:", total_violations)
```

Total Violations: 8023030

Total Violations The analysis shows that:

Total Toll Violations = 8,023,030

This indicates that toll violations are a **large-scale operational issue**, not isolated incidents. The magnitude suggests that even small improvements in enforcement could result in significant revenue recovery.

```
[41]: violations_by_year = traffic_data.groupby("Year")["VIOLATION"].sum().
      ↪reset_index()
```

```

import matplotlib.pyplot as plt

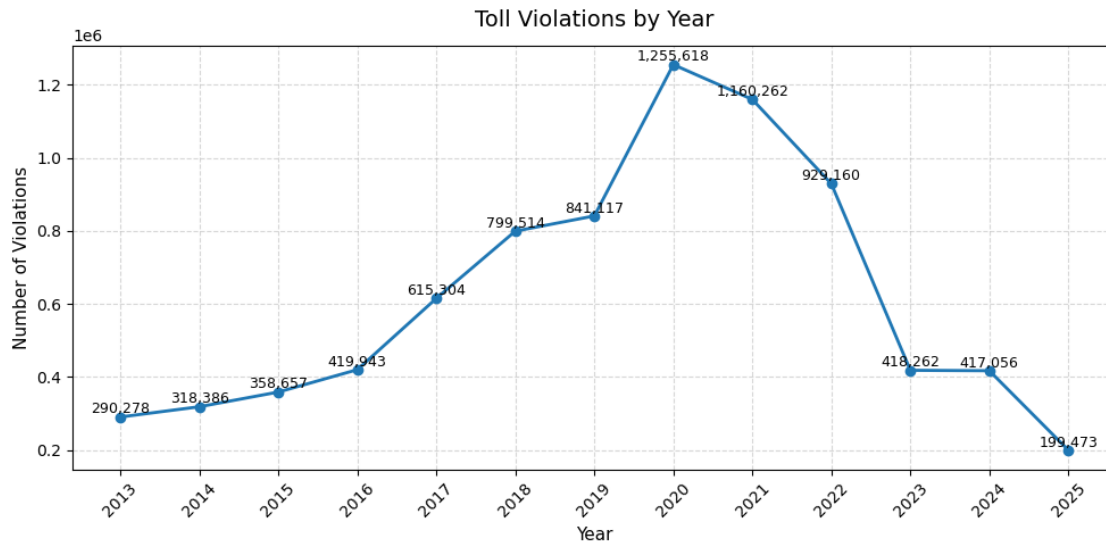
# Create yearly summary
violations_by_year = (
    traffic_data.groupby("Year")["VIOLATION"]
    .sum()
    .reset_index()
)

plt.figure(figsize=(10, 5))
plt.plot(
    violations_by_year["Year"],
    violations_by_year["VIOLATION"],
    marker="o",
    linewidth=2
)

# Add value labels
for x, y in zip(violations_by_year["Year"], violations_by_year["VIOLATION"]):
    plt.text(
        x,
        y + 10000,    # add a vertical buffer (adjust based on scale)
        f"{y:,}",
        ha="center",
        fontsize=9
    )

plt.title("Toll Violations by Year", fontsize=14, pad=12)
plt.xlabel("Year", fontsize=11)
plt.ylabel("Number of Violations", fontsize=11)
plt.xticks(violations_by_year["Year"], rotation=45)
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()

```



From the yearly trend:

- Violations increased steadily from **2013** (~290K) to **2020** (~1.25M)
- Peak occurred around **2020**
- A decline is observed after 2021

Interpretation:

- The steady increase suggests **growing system exploitation over time**
- The spike around 2020 may reflect:
 - changes in travel behavior
 - enforcement gaps
 - increased dependence on toll infrastructure
- The decline after 2022 may be due to:
 - partial data
 - policy or enforcement improvements

Key Insight:

Violations follow long-term trends and are not random events

```
[38]: violations_by_month = traffic_data.groupby("Month")["VIOLATION"].sum().
      ↪reset_index()
      # Create monthly summary
      violations_by_month = (
          traffic_data.groupby("Month")["VIOLATION"]
              .sum()
              .reset_index()
      )

      # Month labels
      month_labels = {
```

```

1: "Jan", 2: "Feb", 3: "Mar", 4: "Apr",
5: "May", 6: "Jun", 7: "Jul", 8: "Aug",
9: "Sep", 10: "Oct", 11: "Nov", 12: "Dec"
}

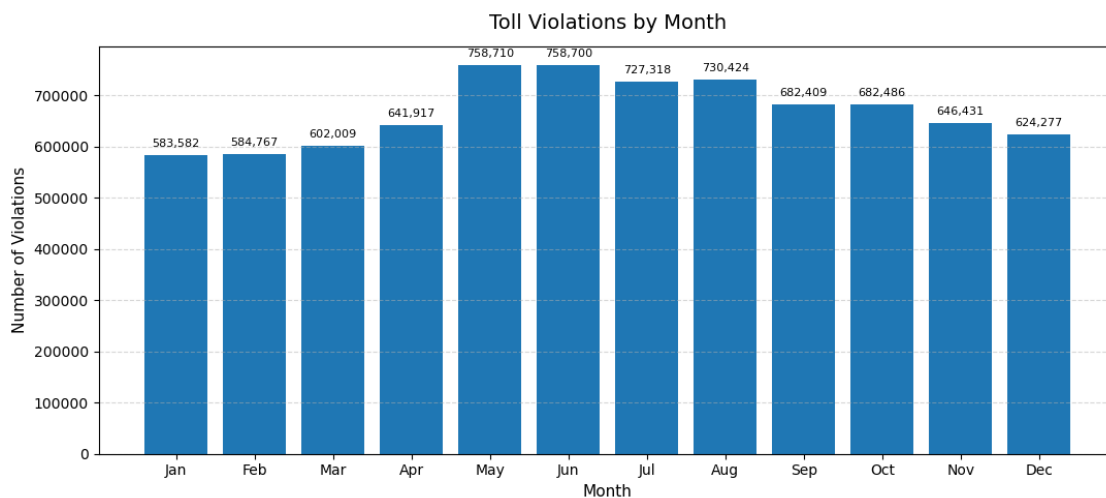
violations_by_month["Month_Name"] = violations_by_month["Month"].
    ↪map(month_labels)

plt.figure(figsize=(11, 5))
bars = plt.bar(
    violations_by_month["Month_Name"],
    violations_by_month["VIOLATION"]
)

# Add labels on top of bars
for bar in bars:
    height = bar.get_height()
    plt.annotate(
        f"{int(height):,}",
        (bar.get_x() + bar.get_width() / 2, height),
        textcoords="offset points",
        xytext=(0, 6),
        ha="center",
        fontsize=8
    )

plt.title("Toll Violations by Month", fontsize=14, pad=12)
plt.xlabel("Month", fontsize=11)
plt.ylabel("Number of Violations", fontsize=11)
plt.grid(axis="y", linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()

```



Violations by Month (Seasonality) Monthly analysis shows:

- Higher violations during **May–August**
- Lower violations during **January–February**

Interpretation:

- Warmer months $\rightarrow$ increased travel $\rightarrow$ more violations
- Winter months $\rightarrow$ reduced travel $\rightarrow$ fewer violations

Key Insight:

Violations are strongly correlated with traffic volume and seasonality

```
[31]: violations_by_week = traffic_data.groupby("Week")["VIOLATION"].sum().  
      ↪reset_index()  
      violations_by_week.head()
```

[31]:	Week	VIOLATION
0	2	64398
1	3	164663
2	4	279817
3	5	75805
4	6	273127

```
[37]: violations_by_fac = traffic_data.groupby("FAC")["VIOLATION"].sum().reset_index()

# Create facility summary and sort descending
violations_by_fac = (
    traffic_data.groupby("FAC")["VIOLATION"]
    .sum()
    .reset_index()
    .sort_values(by="VIOLATION", ascending=False)
)

plt.figure(figsize=(10, 5))
bars = plt.bar(
    violations_by_fac["FAC"].astype(str),
    violations_by_fac["VIOLATION"]
)

# Add labels on top of bars
for bar in bars:
    height = bar.get_height()
    plt.annotate(
        f"{int(height):,}",

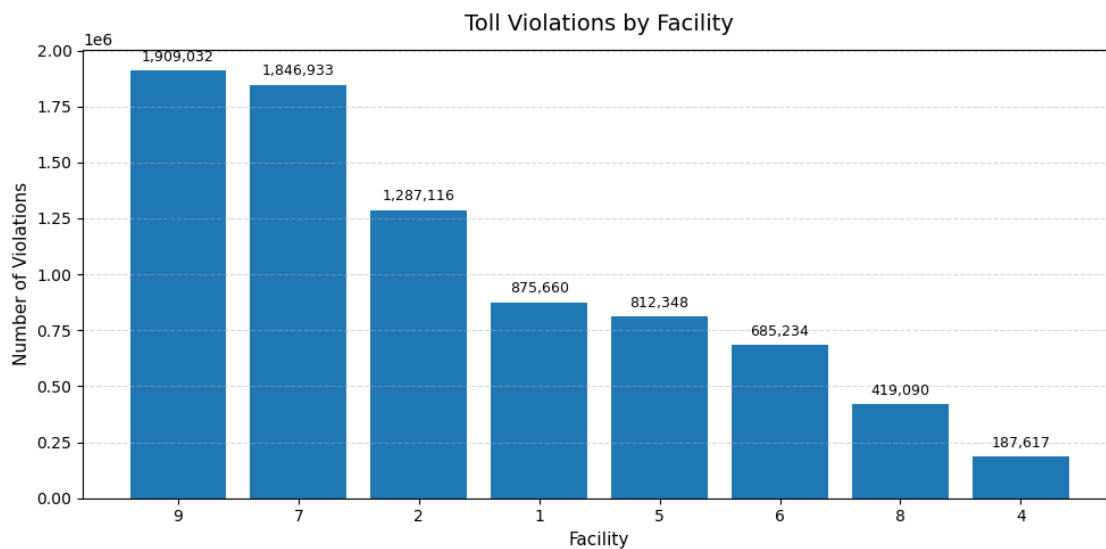
```

```

        (bar.get_x() + bar.get_width() / 2, height),
        textcoords="offset points",
        xytext=(0, 6),
        ha="center",
        fontsize=9
    )

plt.title("Toll Violations by Facility", fontsize=14, pad=12)
plt.xlabel("Facility", fontsize=11)
plt.ylabel("Number of Violations", fontsize=11)
plt.grid(axis="y", linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()

```



```

[33]: violations_by_fac["Percentage"] = (
        violations_by_fac["VIOLATION"] / total_violations * 100
    )

violations_by_fac.sort_values(by="VIOLATION", ascending=False)

```

```

[33]:
   FAC  VIOLATION  Percentage
7    9    1909032    23.794402
5    7    1846933    23.020393
1    2    1287116    16.042767
0    1     875660    10.914330
3    5     812348    10.125202
4    6     685234     8.540838
6    8     419090     5.223588

```

2 4 187617 2.338481

Violations by Facility (Critical Insight) Facility-level breakdown reveals:

- **Facility 9** → **23.79%**
- **Facility 7** → **23.02%**
- **Facility 2** → **16.04%**
- Remaining facilities contribute significantly less

Interpretation:

- Just **two facilities (FAC 9 & 7)** contribute nearly **47% of total violations**
- Violations are **highly concentrated**, not evenly distributed

Key Insight (MOST IMPORTANT):

Violation behavior is facility-driven, not system-wide

4.1.7 Analytical Approach

This question is addressed using **descriptive analytics**, not predictive modeling.

Why no model was used:

- The objective is to answer:
 - “How many?”
 - “Where?”
 - “When?”
- Not to predict future outcomes

Steps performed:

1. Data Preparation

- Converted **DATE** to datetime format
- Extracted:
 - Year
 - Month
 - Week

2. Aggregation

- Total violations calculated using sum
- Grouped violations by:
 - Year
 - Month
 - Week
 - Facility

3. Percentage Contribution

- Facility-level contribution calculated to identify concentration

4. Visualization

- Line chart → trend over time
- Bar charts → facility and seasonal comparison

This approach ensures that the analysis remains **transparent, interpretable, and business-focused**.

4.1.8 Key Insights Summary

From the analysis, three major patterns emerge:

1. Violations are Increasing Over Time

- Indicates systemic issues rather than isolated behavior
- Suggests growing adaptation by drivers

2. Violations Follow Traffic Patterns

- Higher during peak travel seasons
- Strong link to demand

3. Violations are Highly Concentrated

- Majority occur in a small number of facilities
- Enables targeted intervention

4.1.9 Business Recommendations

Based on the findings, the following actions are recommended:

1. Target High-Risk Facilities

- Focus enforcement on **FAC 9 and FAC 7**
- Deploy:
 - additional monitoring systems
 - stricter enforcement mechanisms
- This will yield the **highest return on investment**

2. Seasonal Enforcement Strategy

- Increase enforcement during **high-traffic months (May–August)**
- Optimize staffing based on demand patterns

3. Data-Driven Monitoring

- Implement dashboards to track:
 - violations by facility
 - violations by time
- Enable **real-time decision-making**

4. Policy & Pricing Review

- High violations may indicate:
 - toll avoidance behavior
- Consider:

- pricing adjustments
- communication strategies

5. Cross-Agency Coordination

- Align enforcement with traffic patterns and public transit schedules
- Prevent congestion spillover effects

4.1.10 Conclusion

This analysis shows that toll violations across Port Authority facilities are:

- **large in scale (8M+)**
- **increasing over time**
- **seasonally driven**
- **highly concentrated in specific facilities**

The most critical finding is that violations are **not evenly distributed**, meaning targeted interventions can significantly reduce losses.

By focusing on high-risk facilities and peak periods, the Port Authority can improve enforcement efficiency, reduce revenue leakage, and enhance operational effectiveness.

4.2 Project Question 3

4.2.1 What is the question the company is asking?

The Port Authority wants to understand when traffic reaches its peak levels and how it is affected by seasonality, toll violations, vehicle composition, and external factors such as weather and time-based travel patterns.

4.2.2 What is the business answer to this question?

This question focuses on identifying traffic patterns rather than predicting future values. The goal is to determine when traffic is highest and what behavioral and environmental factors contribute to these peaks.

4.2.3 Models and Tools Used

This is a descriptive analytics problem. The objective is to identify patterns across time and key variables, not to build a predictive model. Therefore, Python-based aggregation, correlation analysis, and visualization techniques are used instead of AutoML.

4.2.4 Why a model is NOT used here

The business question asks “when” and “under what conditions” traffic is high, rather than “what will traffic be.” Therefore, descriptive analysis is more appropriate than regression or classification.

4.2.5 What we are doing

We analyze traffic across: - Year (long-term trends) - Month (seasonality) - Day of week (behavioral patterns) - Weekday vs weekend (travel behavior) - Busiest days (extreme events) - Relationship with violations - Vehicle composition - Weather conditions

4.2.6 Why we are doing this

Understanding traffic patterns helps the Port Authority: - Plan staffing and operations - Manage congestion - Align enforcement strategies - Anticipate peak demand periods

```
[43]: # Ensure DATE is datetime
      traffic_data["DATE"] = pd.to_datetime(traffic_data["DATE"])

      # Create time-based features
      traffic_data["Year"] = traffic_data["DATE"].dt.year
      traffic_data["Month"] = traffic_data["DATE"].dt.month
      traffic_data["Week"] = traffic_data["DATE"].dt.isocalendar().week
      traffic_data["Day"] = traffic_data["DATE"].dt.day
```

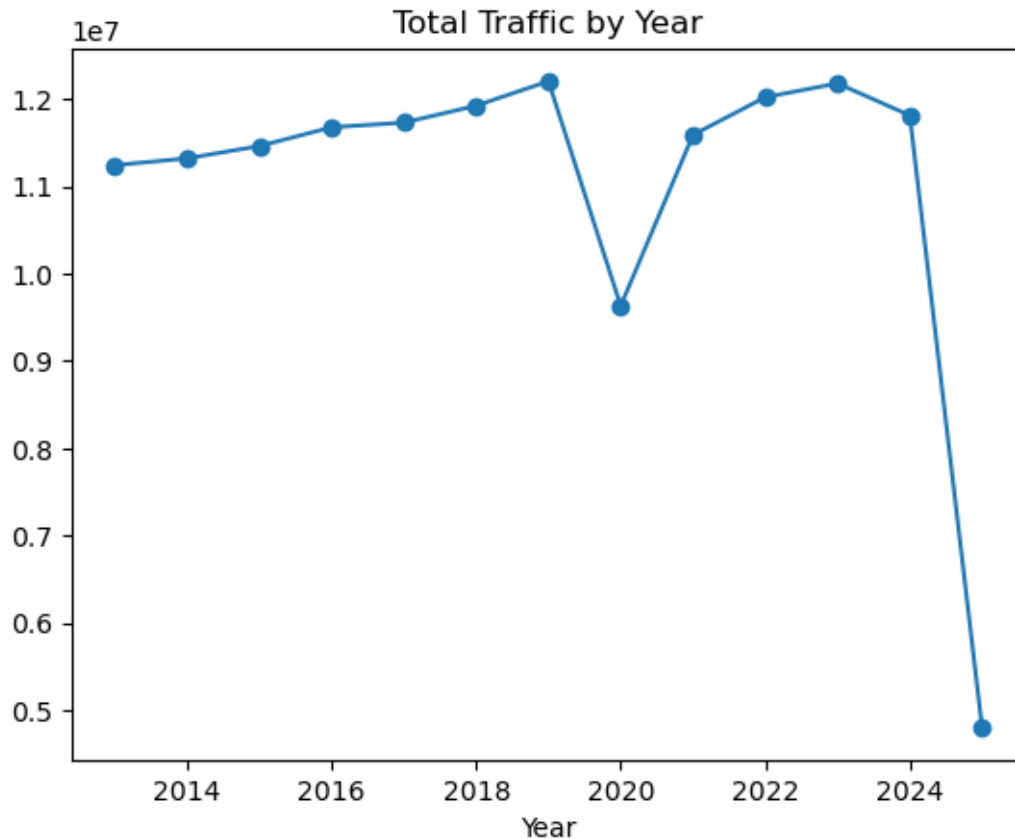
```
[59]: # Traffic by Year
      traffic_by_year = traffic_data.groupby("Year")["TOTAL"].sum()

      print(traffic_by_year)

      traffic_by_year.plot(title="Total Traffic by Year", marker='o')
```

```
Year
2013    11245850
2014    11323991
2015    11464737
2016    11680453
2017    11733732
2018    11927417
2019    12208156
2020     9633298
2021    11592347
2022    12024689
2023    12183377
2024    11816208
2025     4802818
Name: TOTAL, dtype: int64
```

```
[59]: <Axes: title={'center': 'Total Traffic by Year'}, xlabel='Year'>
```



The yearly trend shows that traffic increased steadily from 2013 through 2019, rising from approximately 11.2 million to over 12.2 million vehicles. This indicates a **consistent growth in demand** for bridge and tunnel usage over time.

A sharp decline is observed in 2020, where traffic drops significantly to around 9.6 million. This represents a **major external disruption**, rather than a normal fluctuation. Traffic then recovers in the following years, reaching over 12.1 million again by 2023. The lower value in 2025 reflects partial-year data rather than a true decline.

Overall, this shows that traffic demand is **structurally increasing over time**, while remaining sensitive to external shocks.

```
[60]: # Traffic by Month
traffic_by_month = traffic_data.groupby("Month")["TOTAL"].mean()

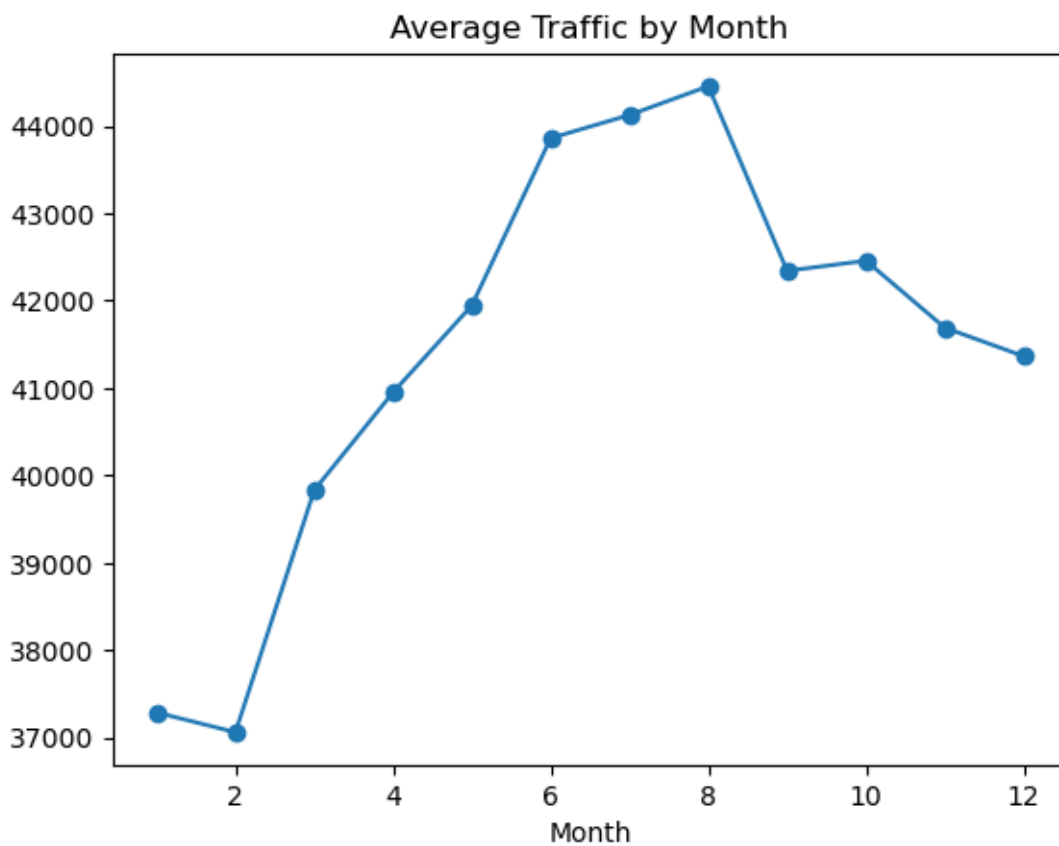
print(traffic_by_month)

traffic_by_month.plot(title="Average Traffic by Month", marker='o')
```

```
Month
1      37287.669967
2      37061.450331
```

```
3    39831.785479
4    40951.386139
5    41946.541254
6    43860.315603
7    44126.007143
8    44455.261649
9    42342.605735
10   42458.340502
11   41691.584229
12   41365.315412
Name: TOTAL, dtype: float64
```

```
[60]: <Axes: title={'center': 'Average Traffic by Month'}, xlabel='Month'>
```



The monthly analysis reveals a clear **seasonal pattern**. Traffic is lowest in the early months of the year, particularly January and February, and gradually increases toward the summer months.

Peak traffic occurs between **June and August**, where average daily traffic exceeds 44,000 vehicles. After August, traffic begins to decline toward the end of the year.

This pattern indicates that traffic is strongly influenced by **seasonality**, with higher travel demand during warmer months and lower demand during colder periods.

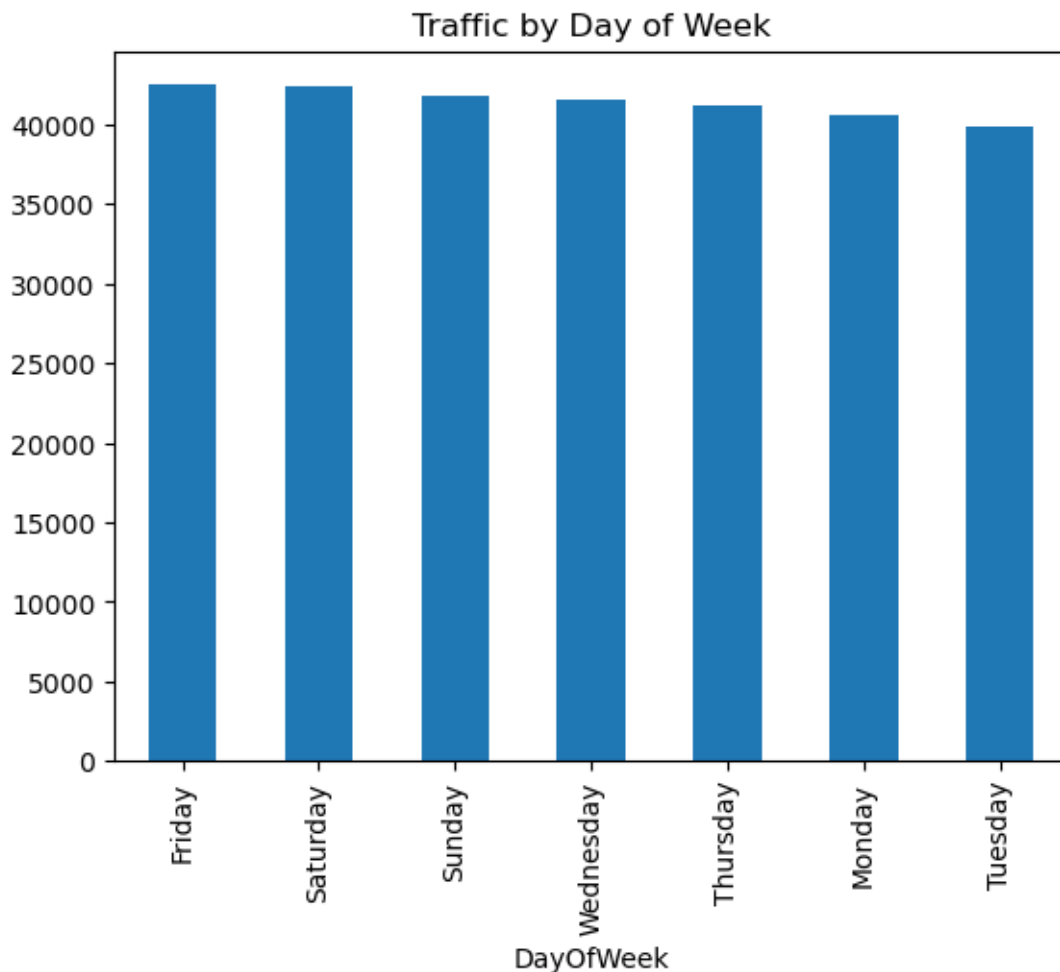
```
[62]: # Traffic by Day of Week
traffic_by_day = traffic_data.groupby("DayOfWeek")["TOTAL"].mean().
    ↪sort_values(ascending=False)

print(traffic_by_day)

traffic_by_day.plot(kind="bar", title="Traffic by Day of Week")
```

```
DayOfWeek
Friday      42485.630480
Saturday    42347.004024
Sunday      41767.769539
Wednesday   41553.004098
Thursday    41188.029014
Monday      40561.288032
Tuesday     39817.441767
Name: TOTAL, dtype: float64
```

```
[62]: <Axes: title={'center': 'Traffic by Day of Week'}, xlabel='DayOfWeek'>
```



The day-of-week analysis shows that traffic is relatively stable throughout the week but follows a consistent pattern. Traffic is slightly higher toward the end of the week, particularly on **Fridays and Saturdays**, and lowest on Tuesdays.

This reflects a combination of **commuting behavior and leisure travel**, where weekday traffic is driven by work-related movement and weekend traffic includes discretionary travel.

```
[64]: # Weekday vs Weekend
traffic_data["Is_Weekend"] = traffic_data["DayOfWeek"].isin(["Saturday", "Sunday"])

weekday_weekend = traffic_data.groupby("Is_Weekend")["TOTAL"].mean()
weekday_weekend.index = ["Weekday", "Weekend"]

print(weekday_weekend)
```

```
Weekday    41110.503030
Weekend    42056.805221
Name: TOTAL, dtype: float64
```

The comparison between weekday and weekend traffic shows that **weekend traffic (~42,056)** is slightly higher than **weekday traffic (~41,110)**.

This difference is not large, but it is meaningful. It suggests that traffic is not driven exclusively by commuting patterns. Instead, **leisure and discretionary travel during weekends** plays an important role in overall traffic levels.

At the same time, the relatively small gap confirms that **commuting demand remains a strong contributor**, resulting in a balanced traffic structure across the week.

```
[65]: # Busiest Days
busiest_days = traffic_data.sort_values(by="TOTAL", ascending=False).head(10)

busiest_days["DayOfWeek"] = busiest_days["DATE"].dt.day_name()

print(busiest_days[["DATE", "TOTAL", "FAC", "DayOfWeek"]])
```

	DATE	TOTAL	FAC	DayOfWeek
2584	2022-09-24	95214	7	Saturday
2628	2023-12-16	95182	7	Saturday
2606	2023-05-28	93844	7	Sunday
2661	2024-11-16	92482	7	Saturday
2646	2024-06-16	92479	7	Sunday
2611	2023-06-24	92408	7	Saturday
2680	2025-05-24	90519	7	Saturday
2621	2023-10-28	90250	7	Saturday
2679	2025-05-16	89825	7	Friday
2590	2022-11-24	89751	7	Thursday

The analysis of the top 10 busiest days shows that peak traffic events are **highly concentrated in specific facilities**, particularly Facility 7.

The maximum observed traffic exceeds **95,000 vehicles in a single day**, which is significantly higher than average levels. These peak days are not randomly distributed and tend to occur during **high-demand periods**.

This indicates that extreme congestion is **location-specific and event-driven**, rather than uniform across all facilities.

```
[66]: # Correlation
traffic_violation_corr = traffic_data[["TOTAL", "VIOLATION"]].corr()

print(traffic_violation_corr)
```

	TOTAL	VIOLATION
TOTAL	1.000000	0.479997
VIOLATION	0.479997	1.000000

The correlation between traffic and violations is approximately **0.48**, indicating a **moderate positive relationship**.

This suggests that as traffic increases, violations also tend to increase. Higher traffic volumes create more opportunities for toll evasion, leading to increased enforcement pressure during peak periods.

```
[68]: vehicle_cols = ["Autos", "Small_T", "Large_T", "Buses"]

vehicle_corr = traffic_data[vehicle_cols + ["TOTAL"]].corr()

print(vehicle_corr)
```

	Autos	Small_T	Large_T	Buses	TOTAL
Autos	1.000000	0.358496	0.318724	0.164755	0.977927
Small_T	0.358496	1.000000	0.858487	0.375090	0.537942
Large_T	0.318724	0.858487	1.000000	-0.031142	0.469668
Buses	0.164755	0.375090	-0.031142	1.000000	0.267658
TOTAL	0.977927	0.537942	0.469668	0.267658	1.000000

Passenger vehicles (**Autos**) show an extremely strong correlation with total traffic (~0.98), indicating that they are the **primary driver of traffic variation**.

Trucks contribute moderately, while buses have a smaller impact. This suggests that congestion is largely driven by **passenger vehicle flow**.

```
[67]: monthly_analysis = traffic_data.groupby("Month")[["TOTAL", "VIOLATION"]].sum()

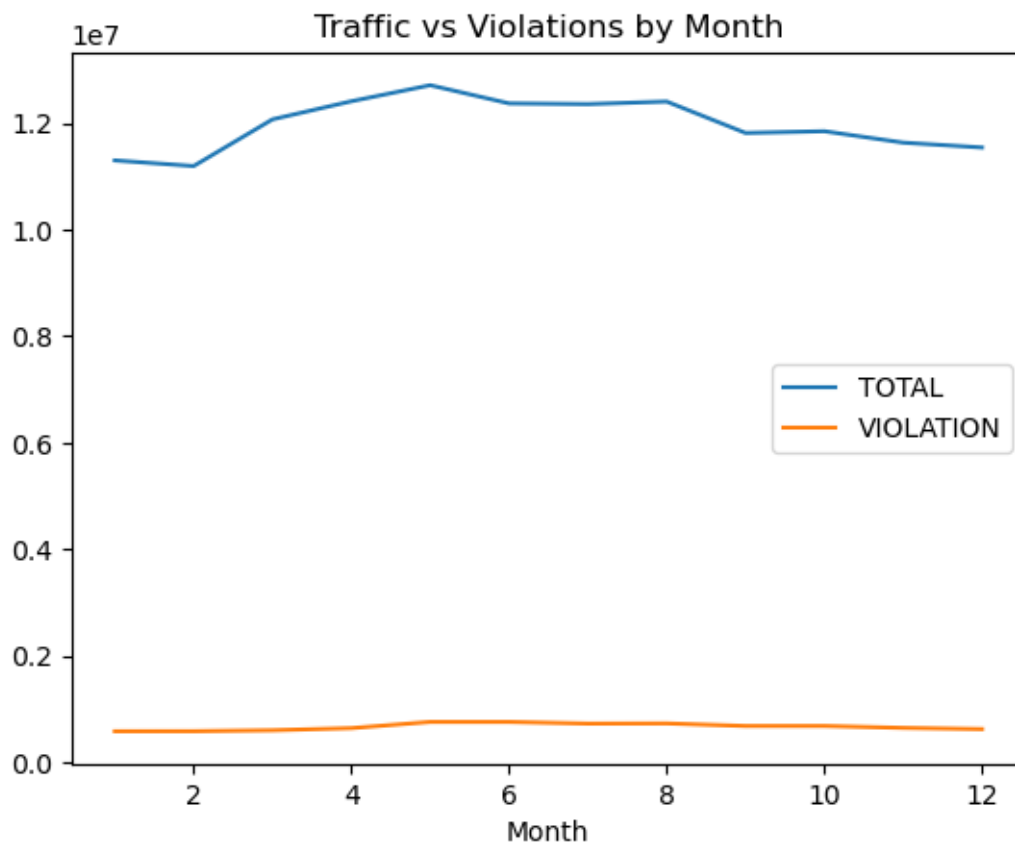
print(monthly_analysis)

monthly_analysis.plot(title="Traffic vs Violations by Month")
```

	TOTAL	VIOLATION
Month		

1	11298164	583582
2	11192558	584767
3	12069031	602009
4	12408270	641917
5	12709802	758710
6	12368609	758700
7	12355282	727318
8	12403018	730424
9	11813587	682409
10	11845877	682486
11	11631952	646431
12	11540923	624277

[67]: <Axes: title={'center': 'Traffic vs Violations by Month'}, xlabel='Month'>



Both traffic and violations follow similar patterns across months, with peaks during **late spring and summer**.

This reinforces the conclusion that violations are **partly driven by traffic volume**, rather than occurring independently.

```
[71]: weather_cols = ["AWND", "PRCP", "SNOW", "TMAX", "TMIN", "TAVG"]

weather_corr = traffic_data[weather_cols + ["TOTAL"]].corr()

print(weather_corr["TOTAL"])

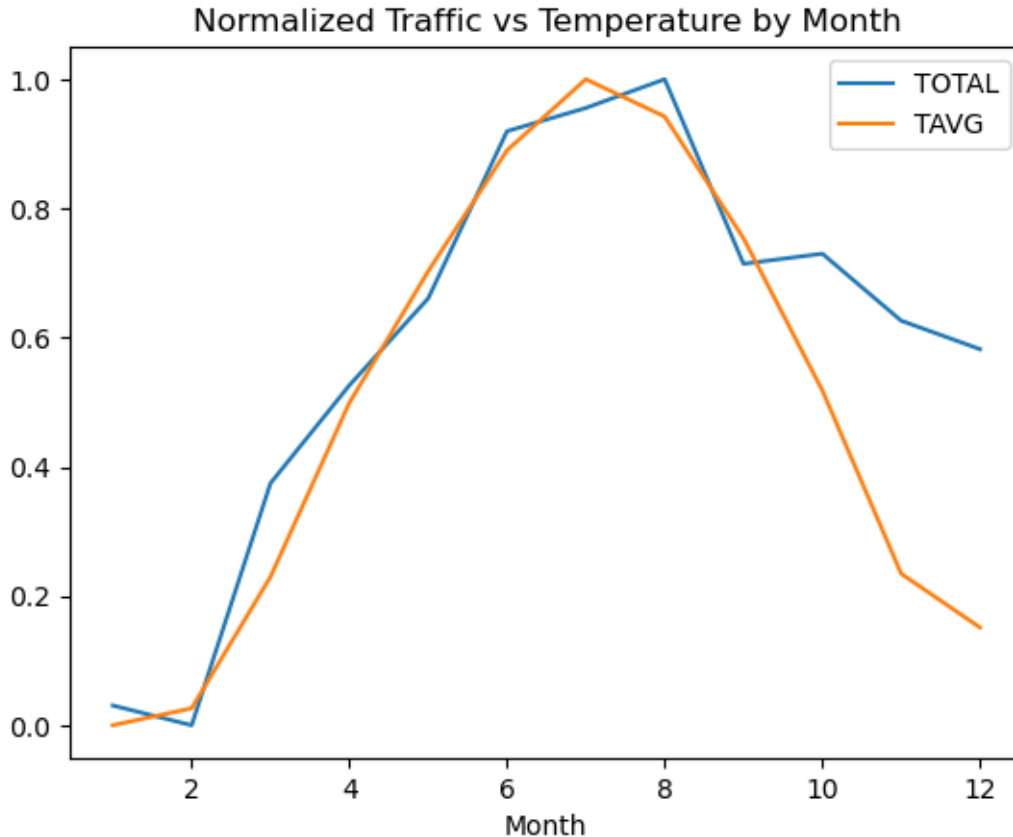
monthly_weather = traffic_data.groupby("Month")[["TOTAL", "TAVG"]].mean()

# Normalize
normalized = (monthly_weather - monthly_weather.min()) / (monthly_weather.max() -
↳ monthly_weather.min())

normalized.plot(title="Normalized Traffic vs Temperature by Month")
```

```
AWND    -0.084962
PRCP    -0.043399
SNOW    -0.114527
TMAX     0.116532
TMIN     0.116946
TAVG     0.117915
TOTAL    1.000000
Name: TOTAL, dtype: float64
```

```
[71]: <Axes: title={'center': 'Normalized Traffic vs Temperature by Month'},
      xlabel='Month'>
```



The normalized comparison between traffic and average temperature shows that both variables follow a similar seasonal pattern. Traffic and temperature increase together from the beginning of the year, peak during the summer months, and decline toward the end of the year.

This indicates a clear directional relationship, where warmer conditions are associated with higher traffic levels. However, the statistical correlation between temperature and traffic is relatively weak (approximately 0.11–0.12), suggesting that temperature alone does not strongly explain variations in traffic.

This apparent difference between the visual trend and the correlation result highlights that temperature is closely aligned with broader seasonal patterns rather than acting as an independent driver of traffic. In other words, weather reinforces seasonality but is not the primary factor influencing traffic behavior.

Additionally, adverse weather conditions such as snow show a negative relationship with traffic, indicating reduced travel during such periods.

Overall, weather plays a supporting role in traffic patterns, while time-based factors such as seasonality and travel behavior remain the dominant drivers.

4.2.7 Business Answer

Traffic across Port Authority bridges and tunnels follows clear and structured patterns driven by time, behavior, and environmental conditions. Traffic is not randomly distributed; instead, it exhibits strong seasonal trends, weekly patterns, and concentration in specific facilities and time periods.

4.2.8 Key Insights

- Traffic shows a **long-term increasing trend**, with a temporary disruption in 2020 followed by recovery
- **Seasonality is a major driver**, with peak traffic during summer months (June–August)
- **Weekend traffic is slightly higher than weekday traffic**, indicating the importance of leisure and discretionary travel alongside commuting
- Peak traffic events are **not uniformly distributed**, but concentrated in specific facilities (especially Facility 7) and specific time periods
- There is a **moderate relationship between traffic and violations**, suggesting that higher traffic leads to increased enforcement pressure
- Passenger vehicles (**Autos**) are the **dominant contributor to traffic variation**
- Weather shows a **supporting but limited role**, reinforcing seasonal patterns rather than independently driving traffic

4.2.9 Recommendations

1. Increase staffing and operational capacity during **peak summer months**, when traffic is highest
2. Focus congestion management efforts on **weekends and late-week periods**, where demand is slightly higher
3. Strengthen enforcement strategies during **high-traffic periods**, as violations increase with traffic volume
4. Prioritize high-demand facilities such as **Facility 7**, where peak traffic events are concentrated
5. Focus traffic management policies on **passenger vehicle flow**, as it is the primary driver of congestion
6. Incorporate **seasonal and weather trends** into operational planning to improve efficiency and preparedness

4.2.10 Models and Tools Used

This question is addressed using **descriptive analytics techniques in Python**, including aggregation, correlation analysis, and visualization. No predictive model was used because the objective is to identify patterns rather than forecast outcomes. This ensures that the insights remain directly aligned with the business question and are easily interpretable for decision-making.

4.3 Project Question 4

4.3.1 What are we analyzing?

In this section, we evaluate whether congestion pricing has led to a **shift in traffic patterns across facilities in 2025**.

The goal is to determine whether drivers are **changing their route choices**, potentially avoiding certain facilities and increasing congestion in others.

4.3.2 Why this matters

Understanding how traffic redistributes across facilities allows us to:

- Detect **behavioral shifts caused by pricing or congestion**
- Identify facilities experiencing **increased or reduced demand**
- Support **data-driven traffic management and policy decisions**

4.3.3 Analytical approach

This analysis is conducted in two parts:

1. Descriptive Analysis (Primary Focus)

We compare traffic patterns before 2025 and during 2025 using **facility-level traffic share** to identify shifts in driver behavior.

2. Model-Based Analysis (Supporting)

We build a classification model using H2O AutoML to identify the key factors associated with **high traffic (congestion conditions)** and support the observed patterns.

4.3.4 Key idea

If traffic share increases in some facilities and decreases in others, it indicates that drivers are **rerouting**, which provides direct evidence of congestion or pricing impact.

The model further helps validate and explain the conditions under which congestion occurs.

```
[72]: # -----  
# STEP 1: Ensure datetime + extract year  
# -----  
traffic_data["DATE"] = pd.to_datetime(traffic_data["DATE"])  
traffic_data["Year"] = traffic_data["DATE"].dt.year  
  
# -----  
# STEP 2: Split data (Before vs 2025)
```

```

# -----
before_2025 = traffic_data[traffic_data["Year"] < 2025]
data_2025 = traffic_data[traffic_data["Year"] == 2025]

# -----
# STEP 3: Traffic by facility
# -----
before_fac = (
    before_2025.groupby("FAC")["TOTAL"]
        .sum()
        .reset_index()
)

after_fac = (
    data_2025.groupby("FAC")["TOTAL"]
        .sum()
        .reset_index()
)

# -----
# STEP 4: Convert to share (%)
# -----
before_fac["Share_Before"] = before_fac["TOTAL"] / before_fac["TOTAL"].sum() * 100
after_fac["Share_2025"] = after_fac["TOTAL"] / after_fac["TOTAL"].sum() * 100

# Merge
comparison = before_fac.merge(after_fac, on="FAC", suffixes=("_Before", "_2025"))

# Change in share
comparison["Change_%"] = comparison["Share_2025"] - comparison["Share_Before"]

# Sort descending (most increase first)
comparison = comparison.sort_values(by="Change_%", ascending=False)

print("Traffic Share Comparison (Before vs 2025):")
print(comparison)

# -----
# STEP 5: Visualization
# -----
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))

plt.bar(

```

```

comparison["FAC"].astype(str),
comparison["Change_%"]
)

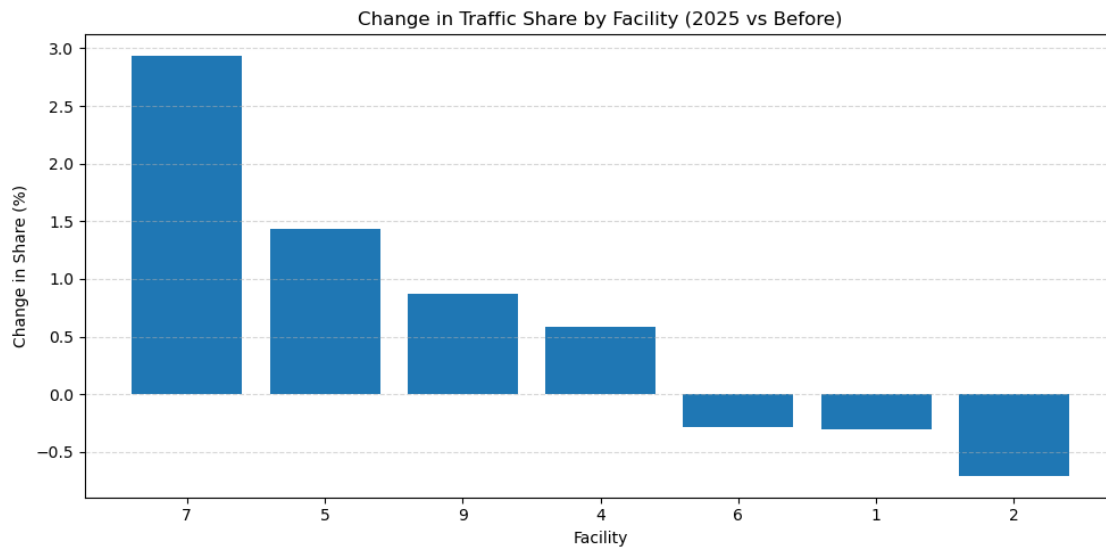
plt.title("Change in Traffic Share by Facility (2025 vs Before)")
plt.xlabel("Facility")
plt.ylabel("Change in Share (%)")

plt.grid(axis="y", linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()

```

Traffic Share Comparison (Before vs 2025):

	FAC	TOTAL_Before	Share_Before	TOTAL_2025	Share_2025	Change_%
5	7	28302792	20.386029	1120154	23.322849	2.936820
3	5	19188771	13.821352	732711	15.255856	1.434504
6	9	24553700	17.685621	891429	18.560541	0.874920
2	4	3701987	2.666480	156398	3.256380	0.589900
4	6	17320148	12.475414	585613	12.193112	-0.282301
0	1	17664692	12.723583	596460	12.418959	-0.304624
1	2	21800782	15.702740	720053	14.992302	-0.710437



4.3.5 What we observe

The comparison of traffic share across facilities clearly shows that traffic distribution has shifted in 2025 relative to previous years.

The most significant increase is observed in **Facility 7 (+2.94%)**, followed by **Facility 5 (+1.43%)**, **Facility 9 (+0.87%)**, and **Facility 4 (+0.59%)**.

On the other hand, facilities such as **Facility 2 (-0.71%)**, **Facility 1 (-0.30%)**, and **Facility 6 (-0.28%)** show a decline in traffic share.

4.3.6 Key interpretation

These changes indicate that traffic is **not evenly distributed across facilities** and that drivers are adjusting their route choices.

The increase in traffic share in certain facilities suggests that drivers may be: - **Rerouting toward alternative crossings** - Avoiding specific facilities that may be more congested or costly

At the same time, the decline in other facilities indicates potential **avoidance behavior**, where drivers are shifting away from those routes.

4.3.7 Why this matters

This shift is critical because it shows that congestion pricing or congestion-related factors do not reduce traffic uniformly. Instead, traffic is **redistributed across the network**.

This creates a situation where: - Some facilities experience **reduced pressure** - Others experience **increased congestion**

4.3.8 What this means for congestion

The strong increase in Facility 7 suggests that it is absorbing a larger portion of total traffic and may face **higher congestion risk**.

Similarly, facilities with declining share may be experiencing **reduced demand**, possibly due to pricing sensitivity or congestion avoidance.

This confirms that congestion is: - **Dynamic** - **Facility-specific** - Influenced by driver decision-making

4.3.9 Conclusion

The results provide clear evidence that traffic patterns in 2025 differ from historical patterns.

The observed redistribution across facilities indicates that drivers are **adapting their behavior**, likely in response to congestion or pricing conditions.

Rather than eliminating congestion, these changes suggest that congestion is being **shifted across facilities**, which has important implications for traffic management and policy design.

To further support these findings, a classification model is developed in the next step to identify the key factors associated with high traffic (congestion conditions). This helps validate whether the observed shifts are consistent with underlying traffic drivers such as facility, time patterns, and environmental conditions.

4.3.10 Model-Based Analysis: Identifying Drivers of Congestion

After identifying a shift in traffic patterns across facilities, we now build a model to understand the **underlying drivers of congestion**.

4.3.11 Why we are building a model

The descriptive analysis shows that traffic is being redistributed across facilities.

However, to fully understand this behavior, we need to identify:

- What conditions lead to **high traffic (congestion)**
- Whether congestion is influenced by **facility, time, or external factors**
- Whether the observed shift aligns with known traffic drivers

This allows us to move from **observation** → **explanation**.

4.3.12 Modeling approach

We treat congestion as a **classification problem**, where:

- High Traffic (1) → Congestion condition
- Normal Traffic (0) → Non-congested condition

4.3.13 Key decisions

1. Target Variable

We define congestion using the **75th percentile of traffic (TOTAL)**.

This ensures we focus on **peak traffic conditions**, rather than arbitrary thresholds.

2. Model Selection (AutoML)

We use H2O AutoML because:

- It evaluates multiple algorithms (GBM, Random Forest, Deep Learning, etc.)
- It selects the best-performing model objectively
- It avoids bias in model selection

3. Avoiding Data Leakage

We exclude variables such as:

- TOTAL
- CASH
- EZPASS
- VIOLATION
- Vehicle counts

These are direct components of traffic and would artificially inflate model performance.

4. Features Used

We include:

- Facility (FAC)

- Time patterns (DayOfWeek)
- Weather variables (TAVG, PRCP, SNOW, etc.)
- Passenger and bus volumes

These represent **true drivers of traffic behavior**, not derived outcomes.

4.3.14 Objective

The goal of this model is not just prediction, but to identify:

- Which factors most strongly influence congestion
- Whether facility-level patterns align with observed traffic shifts

```
[73]: # -----
# STEP 1: Create target variable
# -----
threshold = traffic_data["TOTAL"].quantile(0.75)

traffic_data["High_Traffic"] = (traffic_data["TOTAL"] >= threshold).astype(int)

print("High Traffic Threshold:", threshold)
traffic_data[["TOTAL", "High_Traffic"]].head()

# -----
# STEP 2: Prepare dataset
# -----
question4_data = traffic_data.copy()

# Drop unnecessary columns
if "STATION" in question4_data.columns:
    question4_data = question4_data.drop(columns=["STATION"])

# Convert date fields
question4_data["DATE"] = pd.to_datetime(question4_data["DATE"])
question4_data["Month_Year"] = pd.to_datetime(question4_data["Month_Year"])
```

High Traffic Threshold: 54270.5

```
[74]: # -----
# STEP 3: Convert to H2O frame
# -----
import h2o
from h2o.automl import H2OAutoML

# (Safe - won't restart if already running)
```

```

h2o.init()

question4_h2o = h2o.H2OFrame(question4_data)

# Convert categorical variables
question4_h2o["FAC"] = question4_h2o["FAC"].asfactor()
question4_h2o["DayOfWeek"] = question4_h2o["DayOfWeek"].asfactor()
question4_h2o["High_Traffic"] = question4_h2o["High_Traffic"].asfactor()

# -----
# STEP 4: Define target & predictors
# -----
target_q4 = "High_Traffic"

predictors_q4 = [
    "FAC",
    "AWND",
    "PRCP",
    "SNOW",
    "TMAX",
    "TMIN",
    "TAVG",
    "Passenger_Volume",
    "Bus_Volume",
    "DayOfWeek"
]

# -----
# STEP 5: Train-test split
# -----
train_q4, test_q4 = question4_h2o.split_frame(ratios=[0.8], seed=42)

print("Train rows:", train_q4.nrows)
print("Test rows:", test_q4.nrows)

# -----
# STEP 6: Run AutoML
# -----
automl_q4 = H2OAutoML(
    max_models=15,
    seed=42,
    nfolds=5,
    sort_metric="AUC"
)

```

```

automl_q4.train(
    x=predictors_q4,
    y=target_q4,
    training_frame=train_q4,
    leaderboard_frame=test_q4
)

```

Checking whether there is an H2O instance running at http://localhost:54321.
connected.

```

-----
H2O_cluster_uptime:      4 hours 43 mins
H2O_cluster_timezone:    America/New_York
H2O_data_parsing_timezone: UTC
H2O_cluster_version:     3.46.0.10
H2O_cluster_version_age: 17 days
H2O_cluster_name:        H2O_from_python_nanda_vygxxk
H2O_cluster_total_nodes: 1
H2O_cluster_free_memory: 3.551 Gb
H2O_cluster_total_cores: 8
H2O_cluster_allowed_cores: 8
H2O_cluster_status:      locked, healthy
H2O_connection_url:       http://localhost:54321
H2O_connection_proxy:     {"http": null, "https": null}
H2O_internal_security:    False
Python_version:           3.13.9 final
-----

```

```

Parse progress:
|                                     | (done) 100%
Train rows: 2784
Test rows: 687
AutoML progress: |
02:56:40.697: AutoML: XGBoost is not available; skipping it.

```

```

| (done) 100%

```

[74]: Model Details

```

=====

```

```

H2OStackedEnsembleEstimator : Stacked Ensemble
Model Key: StackedEnsemble_AllModels_1_AutoML_2_20260330_25640

```

Model Summary for Stacked Ensemble:

key	value
Stacking strategy	cross_validation

Number of base models (used / total)	9/15
# GBM base models (used / total)	5/8
# GLM base models (used / total)	1/1
# DRF base models (used / total)	0/2
# DeepLearning base models (used / total)	3/4
Metalearner algorithm	GLM
Metalearner fold assignment scheme	Random
Metalearner nolds	5
Metalearner fold_column	
Custom metalearner hyperparameters	None

ModelMetricsBinomialGLM: stackedensemble

** Reported on train data. **

MSE: 0.026516207996117137
 RMSE: 0.16283798081564735
 LogLoss: 0.09633550693049299
 AUC: 0.9951173928199409
 AUCPR: 0.9858713516286243
 Gini: 0.9902347856398819
 Null degrees of freedom: 2783
 Residual degrees of freedom: 2774
 Null deviance: 3142.0203915419106
 Residual deviance: 536.396102588985
 AIC: 556.396102588985

Confusion Matrix (Act/Pred) for max f1 @ threshold = 0.48671333248564624

	0	1	Error	Rate
0	2023	60	0.0288	(60.0/2083.0)
1	29	672	0.0414	(29.0/701.0)
Total	2052	732	0.032	(89.0/2784.0)

Maximum Metrics: Maximum metrics at their respective thresholds

metric	threshold	value	idx
max f1	0.486713	0.937893	201
max f2	0.333599	0.961432	238
max f0point5	0.746669	0.947744	127
max accuracy	0.486713	0.968032	201
max precision	0.99523	1	0
max recall	0.0745932	1	333
max specificity	0.99523	1	0
max absolute_mcc	0.486713	0.916771	201
max min_per_class_accuracy	0.443197	0.965434	211
max mean_per_class_accuracy	0.381264	0.969357	227
max tns	0.99523	2083	0

max fns	0.99523	698	0
max fps	0.000188915	2083	399
max tps	0.0745932	701	333
max tnr	0.99523	1	0
max fnr	0.99523	0.99572	0
max fpr	0.000188915	1	399
max tpr	0.0745932	1	333

Gains/Lift Table: Avg response rate: 25.18 %, avg score: 25.66 %

group	cumulative_data_fraction	lower_threshold	lift	
cumulative_lift	response_rate	score	cumulative_response_rate	
cumulative_score	capture_rate	cumulative_capture_rate	gain	
cumulative_gain	kolmogorov_smirnov			
-----	-----	-----	-----	
-----	-----	-----	-----	
-----	-----	-----	-----	
-----	-----	-----	-----	
1	0.0100575	0.988534	3.97147	3.97147
1	0.99136	1	0.99136	
0.0399429	0.0399429	297.147	297.147	
0.0399429				
2	0.0201149	0.985553	3.97147	3.97147
1	0.987201	1	0.989281	
0.0399429	0.0798859	297.147	297.147	
0.0798859				
3	0.0301724	0.982742	3.97147	3.97147
1	0.984107	1	0.987556	
0.0399429	0.119829	297.147	297.147	0.119829
4	0.0402299	0.979109	3.97147	3.97147
1	0.98116	1	0.985957	
0.0399429	0.159772	297.147	297.147	0.159772
5	0.0502874	0.974908	3.97147	3.97147
1	0.977131	1	0.984192	
0.0399429	0.199715	297.147	297.147	0.199715
6	0.100216	0.950974	3.97147	3.97147
1	0.963374	1	0.97382	
0.198288	0.398003	297.147	297.147	0.398003
7	0.150144	0.899819	3.97147	3.97147
1	0.928967	1	0.958905	
0.198288	0.596291	297.147	297.147	0.596291
8	0.200072	0.790789	3.7429	3.91443
0.942446	0.850963	0.985637	0.931968	
0.186876	0.783167	274.29	291.443	0.779326
9	0.299928	0.282444	2.07145	3.30084
0.521583	0.560714	0.831138	0.808365	
0.206847	0.990014	107.145	230.084	0.922323
10	0.400144	0.028966	0.0996426	2.4991

0.0250896	0.118674	0.629264	0.635632
0.00998573	1	-90.0357	149.91
11	0.5	0.00566326	0
0	0.0146588	0.503592	0.511616
1		-100	100
12	0.599856	0.00212975	0
0	0.00286565	0.41976	0.426926
1		-100	66.7066
13	0.700072	0.00151488	0
0	0.00179675	0.359672	0.366069
1		-100	42.8425
14	0.799928	0.00116206	0
0	0.00132877	0.314773	0.320537
1		-100	25.0112
15	0.899784	0.000793094	0
0	0.000984439	0.27984	0.285074
1		-100	11.1377
16	1	5.97399e-06	0
0	0.000526364	0.251796	0.256558
1		-100	0

ModelMetricsBinomialGLM: stackedensemble

** Reported on cross-validation data. **

MSE: 0.051491437184804964

RMSE: 0.22691724743792607

LogLoss: 0.1676095922537137

AUC: 0.9767707198344316

AUCPR: 0.9259980241616105

Gini: 0.9535414396688633

Null degrees of freedom: 2783

Residual degrees of freedom: 2775

Null deviance: 3142.949700278179

Residual deviance: 933.2502096686778

AIC: 951.2502096686778

Confusion Matrix (Act/Pred) for max f1 @ threshold = 0.39669851354642294

	0	1	Error	Rate
0	1951	132	0.0634	(132.0/2083.0)
1	66	635	0.0942	(66.0/701.0)
Total	2017	767	0.0711	(198.0/2784.0)

Maximum Metrics: Maximum metrics at their respective thresholds

metric	threshold	value	idx
max f1	0.396699	0.865123	226

max f2	0.141504	0.918177	309
max f0point5	0.644662	0.869365	160
max accuracy	0.493914	0.930675	201
max precision	0.995119	1	0
max recall	0.00540359	1	381
max specificity	0.995119	1	0
max absolute_mcc	0.396699	0.818487	226
max min_per_class_accuracy	0.347417	0.924394	240
max mean_per_class_accuracy	0.186606	0.931232	293
max tns	0.995119	2083	0
max fns	0.995119	700	0
max fps	0.000114858	2083	399
max tps	0.00540359	701	381
max tnr	0.995119	1	0
max fnr	0.995119	0.998573	0
max fpr	0.000114858	1	399
max tpr	0.00540359	1	381

Gains/Lift Table: Avg response rate: 25.18 %, avg score: 25.17 %

group	cumulative_data_fraction	lower_threshold	lift	
cumulative_lift	response_rate	score	cumulative_response_rate	
cumulative_score	capture_rate	cumulative_capture_rate	gain	
cumulative_gain	kolmogorov_smirnov			
-----	-----	-----	-----	
-----	-----	-----	-----	
-----	-----	-----	-----	
-----	-----			
1	0.0100575	0.985547	3.97147	3.97147
1	0.989273	1	0.989273	
0.0399429	0.0399429	297.147	297.147	
0.0399429				
2	0.0201149	0.98099	3.97147	3.97147
1	0.983495	1	0.986384	
0.0399429	0.0798859	297.147	297.147	
0.0798859				
3	0.0301724	0.976258	3.97147	3.97147
1	0.978884	1	0.983884	
0.0399429	0.119829	297.147	297.147	0.119829
4	0.0402299	0.972748	3.68779	3.90055
0.928571	0.974532	0.982143	0.981546	
0.0370899	0.156919	268.779	290.055	0.155959
5	0.0502874	0.968162	3.97147	3.91473
1	0.970168	0.985714	0.97927	
0.0399429	0.196862	297.147	291.473	0.195901
6	0.100216	0.939045	3.7429	3.82912
0.942446	0.954758	0.964158	0.967058	
0.186876	0.383738	274.29	282.912	0.378937

7	0.150144		0.877468	3.37146	3.67693
0.848921	0.908293	0.925837		0.947516	
0.168331	0.552068		237.146	267.693	0.537186
8	0.200072		0.739541	3.34289	3.59357
0.841727	0.818993	0.904847		0.915443	
0.166904	0.718973		234.289	259.357	0.693529
9	0.299928		0.313475	2.17145	3.1201
0.546763	0.530967	0.785629		0.787438	
0.216833	0.935806		117.145	212.01	0.849872
10	0.400144		0.0299505	0.555152	2.47771
0.139785	0.132035	0.623878		0.623293	
0.0556348	0.991441		-44.4848	147.771	0.790289
11	0.5		0.00555331	0.0714293	1.99715
0.0179856	0.014455	0.502874		0.5017	
0.00713267	0.998573		-92.8571	99.7147	0.66636
12	0.599856		0.00249625	0.0142859	1.66707
0.00359712	0.00347904	0.41976		0.418763	
0.00142653	1		-98.5714	66.7066	0.534806
13	0.700072		0.00169786	0	1.42842
0	0.00204609	0.359672		0.35911	0
1		-100	42.8425	0.400864	
14	0.799928		0.00123135	0	1.25011
0	0.00146507	0.314773		0.314464	0
1		-100	25.0112	0.267403	
15	0.899784		0.000802993	0	1.11138
0	0.00101317	0.27984		0.279678	0
1		-100	11.1377	0.133941	
16	1		3.3e-06	0	1
0	0.000493225	0.251796		0.251699	0
1		-100	0	0	

Cross-Validation Metrics Summary:

	mean	sd	cv_1_valid	cv_2_valid
cv_3_valid	cv_4_valid	cv_5_valid		
accuracy	0.9328662	0.00811705	0.93536806	0.9368421
0.92665476	0.92266184	0.94280446		
aic	203.7254	13.743845	208.27948	209.96165
218.71307	199.17491	182.49792		
auc	0.9770401	0.003016955	0.97554725	0.97625977
0.9733716	0.97890157	0.9811203		
err	0.06713376	0.00811705	0.064631954	0.06315789
0.07334526	0.07733813	0.05719557		
err_count	37.4	4.7222877	36.0	36.0
41.0	43.0	31.0		
f0point5	0.8495735	0.023365837	0.8789625	0.84

```

0.84076434    0.82072175    0.8674189
f1              0.87355626    0.008886078    0.87142855    0.875
0.86557376    0.8676923     0.8880866
f2              0.89981735    0.022584027    0.8640227    0.9130435
0.8918919     0.9203655     0.90976334
lift_top_group      3.977788     0.18836762    3.9225352    4.2537313
3.8551724     3.7823129     4.075188
loglikelihood        0.0         0.0         0.0         0.0         0.0
0.0           0.0
---          ---          ---          ---          ---          ---
---          ---
mean_per_class_error 0.07184226    0.011987506    0.08969964    0.06196084
0.078644015    0.065640435    0.06326636
mse              0.051384415    0.0032662398    0.052046135    0.05094728
0.056123905    0.05080771     0.046997048
null_deviance      628.58997    15.686213     632.45715     623.0459
640.2324       643.01495     604.1992
pr_auc            0.9270352     0.011907154    0.9262899     0.9156858
0.91520745     0.9414166     0.93657637
precision          0.8347083     0.035333745    0.884058      0.8181818
0.825          0.7921348     0.8541667
r2              0.72709644    0.015660837    0.7259925     0.71667856
0.7078518     0.73876065     0.7461985
recall           0.9187588     0.03794136     0.85915494     0.9402985
0.91034484     0.9591837     0.924812
residual_deviance  186.5254     14.901966     190.27948     191.96165
204.71307     181.17491     164.49792
rmse             0.22658996    0.007194332    0.22813621     0.22571504
0.23690484     0.22540565     0.21678802
specificity        0.9375567     0.01942386     0.9614458     0.9357798
0.93236715     0.90953547     0.94865525
[24 rows x 8 columns]

```

[tips]

Use `model.explain()` to inspect the model.

--

Use `h2o.display.toggle\_user\_tips()` to switch on/off this section.

```

[76]: # -----
# STEP 7: Leaderboard
# -----
leaderboard_q4 = automl_q4.leaderboard
leaderboard_q4.head(rows=15)

```

```

[76]: model_id          auc    logloss
aucpr    mean_per_class_error    rmse    mse

```

```

-----
-----
StackedEnsemble_AllModels_1_AutoML_2_20260330_25640 0.984661 0.136256
0.945233 0.0586769 0.205423 0.0421986
StackedEnsemble_BestOfFamily_1_AutoML_2_20260330_25640 0.984005 0.139717
0.943906 0.0414556 0.208917 0.0436465
GBM_1_AutoML_2_20260330_25640 0.98336 0.143682
0.938071 0.0506564 0.208986 0.0436753
GBM_3_AutoML_2_20260330_25640 0.982836 0.144907
0.938798 0.0546119 0.212548 0.0451765
GBM_2_AutoML_2_20260330_25640 0.982416 0.14512
0.932094 0.0545025 0.211205 0.0446077
GBM_5_AutoML_2_20260330_25640 0.982053 0.145539
0.934963 0.0504376 0.213969 0.0457829
GBM_grid_1_AutoML_2_20260330_25640_model_1 0.981685 0.147958
0.930711 0.0465914 0.211616 0.0447815
GLM_1_AutoML_2_20260330_25640 0.981333 0.148172
0.936839 0.0613427 0.217398 0.0472621
DRF_1_AutoML_2_20260330_25640 0.980671 0.160113
0.927188 0.0492573 0.21929 0.048088
GBM_grid_1_AutoML_2_20260330_25640_model_2 0.980481 0.150628
0.923777 0.0638128 0.216668 0.0469451
GBM_4_AutoML_2_20260330_25640 0.980234 0.151849
0.924656 0.0463726 0.216071 0.0466866
DeepLearning_grid_1_AutoML_2_20260330_25640_model_1 0.978512 0.344373
0.92725 0.0799632 0.319656 0.10218
DeepLearning_1_AutoML_2_20260330_25640 0.978351 0.15758
0.924883 0.0552453 0.226341 0.0512304
DeepLearning_grid_3_AutoML_2_20260330_25640_model_1 0.976359 0.19461
0.921767 0.0704341 0.239622 0.0574186
GBM_grid_1_AutoML_2_20260330_25640_model_3 0.974937 0.178042
0.93158 0.0723572 0.230469 0.0531161
[15 rows x 7 columns]

```

```

[78]: # -----
# STEP 8: Best model
# -----
best_model_q4 = automl_q4.leader
best_model_q4

```

```

[78]: Model Details
=====
H2OStackedEnsembleEstimator : Stacked Ensemble
Model Key: StackedEnsemble_AllModels_1_AutoML_2_20260330_25640

```

Model Summary for Stacked Ensemble:

key	value
-----	-----
Stacking strategy	cross_validation
Number of base models (used / total)	9/15
# GBM base models (used / total)	5/8
# GLM base models (used / total)	1/1
# DRF base models (used / total)	0/2
# DeepLearning base models (used / total)	3/4
Metalearner algorithm	GLM
Metalearner fold assignment scheme	Random
Metalearner nolds	5
Metalearner fold_column	
Custom metalearner hyperparameters	None

ModelMetricsBinomialGLM: stackedensemble

** Reported on train data. **

MSE: 0.026516207996117137
 RMSE: 0.16283798081564735
 LogLoss: 0.09633550693049299
 AUC: 0.9951173928199409
 AUCPR: 0.9858713516286243
 Gini: 0.9902347856398819
 Null degrees of freedom: 2783
 Residual degrees of freedom: 2774
 Null deviance: 3142.0203915419106
 Residual deviance: 536.396102588985
 AIC: 556.396102588985

Confusion Matrix (Act/Pred) for max f1 @ threshold = 0.48671333248564624

	0	1	Error	Rate
-----	----	---	-----	-----
0	2023	60	0.0288	(60.0/2083.0)
1	29	672	0.0414	(29.0/701.0)
Total	2052	732	0.032	(89.0/2784.0)

Maximum Metrics: Maximum metrics at their respective thresholds

metric	threshold	value	idx
-----	-----	-----	-----
max f1	0.486713	0.937893	201
max f2	0.333599	0.961432	238
max f0point5	0.746669	0.947744	127
max accuracy	0.486713	0.968032	201
max precision	0.99523	1	0
max recall	0.0745932	1	333
max specificity	0.99523	1	0

max absolute_mcc	0.486713	0.916771	201
max min_per_class_accuracy	0.443197	0.965434	211
max mean_per_class_accuracy	0.381264	0.969357	227
max tns	0.99523	2083	0
max fns	0.99523	698	0
max fps	0.000188915	2083	399
max tps	0.0745932	701	333
max tnr	0.99523	1	0
max fnr	0.99523	0.99572	0
max fpr	0.000188915	1	399
max tpr	0.0745932	1	333

Gains/Lift Table: Avg response rate: 25.18 %, avg score: 25.66 %

group	cumulative_data_fraction	lower_threshold	lift	
cumulative_lift	response_rate	score	cumulative_response_rate	
cumulative_score	capture_rate	cumulative_capture_rate	gain	
cumulative_gain	kolmogorov_smirnov			
1	0.0100575	0.988534	3.97147	3.97147
1	0.99136	1	0.99136	
0.0399429	0.0399429	297.147	297.147	
0.0399429				
2	0.0201149	0.985553	3.97147	3.97147
1	0.987201	1	0.989281	
0.0399429	0.0798859	297.147	297.147	
0.0798859				
3	0.0301724	0.982742	3.97147	3.97147
1	0.984107	1	0.987556	
0.0399429	0.119829	297.147	297.147	0.119829
4	0.0402299	0.979109	3.97147	3.97147
1	0.98116	1	0.985957	
0.0399429	0.159772	297.147	297.147	0.159772
5	0.0502874	0.974908	3.97147	3.97147
1	0.977131	1	0.984192	
0.0399429	0.199715	297.147	297.147	0.199715
6	0.100216	0.950974	3.97147	3.97147
1	0.963374	1	0.97382	
0.198288	0.398003	297.147	297.147	0.398003
7	0.150144	0.899819	3.97147	3.97147
1	0.928967	1	0.958905	
0.198288	0.596291	297.147	297.147	0.596291
8	0.200072	0.790789	3.7429	3.91443
0.942446	0.850963	0.985637	0.931968	
0.186876	0.783167	274.29	291.443	0.779326

9	0.299928		0.282444	2.07145	3.30084
0.521583	0.560714	0.831138		0.808365	
0.206847	0.990014		107.145	230.084	0.922323
10	0.400144		0.028966	0.0996426	2.4991
0.0250896	0.118674	0.629264		0.635632	
0.00998573	1		-90.0357	149.91	0.801728
11	0.5		0.00566326	0	2
0		0.0146588	0.503592	0.511616	0
1		-100	100	0.668267	
12	0.599856		0.00212975	0	1.66707
0		0.00286565	0.41976	0.426926	0
1		-100	66.7066	0.534806	
13	0.700072		0.00151488	0	1.42842
0		0.00179675	0.359672	0.366069	0
1		-100	42.8425	0.400864	
14	0.799928		0.00116206	0	1.25011
0		0.00132877	0.314773	0.320537	0
1		-100	25.0112	0.267403	
15	0.899784		0.000793094	0	1.11138
0		0.000984439	0.27984	0.285074	0
1		-100	11.1377	0.133941	
16	1		5.97399e-06	0	1
0		0.000526364	0.251796	0.256558	0
1		-100	0	0	

ModelMetricsBinomialGLM: stackedensemble

** Reported on cross-validation data. **

MSE: 0.051491437184804964

RMSE: 0.22691724743792607

LogLoss: 0.1676095922537137

AUC: 0.9767707198344316

AUCPR: 0.9259980241616105

Gini: 0.9535414396688633

Null degrees of freedom: 2783

Residual degrees of freedom: 2775

Null deviance: 3142.949700278179

Residual deviance: 933.2502096686778

AIC: 951.2502096686778

Confusion Matrix (Act/Pred) for max f1 @ threshold = 0.39669851354642294

	0	1	Error	Rate
0	1951	132	0.0634	(132.0/2083.0)
1	66	635	0.0942	(66.0/701.0)
Total	2017	767	0.0711	(198.0/2784.0)

Maximum Metrics: Maximum metrics at their respective thresholds

Gains/Lift Table: Avg response rate: 25.18 %, avg score: 25.17 %

0.0399429	0.196862		297.147	291.473	0.195901
6	0.100216		0.939045	3.7429	3.82912
0.942446	0.954758	0.964158		0.967058	
0.186876	0.383738		274.29	282.912	0.378937
7	0.150144		0.877468	3.37146	3.67693
0.848921	0.908293	0.925837		0.947516	
0.168331	0.552068		237.146	267.693	0.537186
8	0.200072		0.739541	3.34289	3.59357
0.841727	0.818993	0.904847		0.915443	
0.166904	0.718973		234.289	259.357	0.693529
9	0.299928		0.313475	2.17145	3.1201
0.546763	0.530967	0.785629		0.787438	
0.216833	0.935806		117.145	212.01	0.849872
10	0.400144		0.0299505	0.555152	2.47771
0.139785	0.132035	0.623878		0.623293	
0.0556348	0.991441		-44.4848	147.771	0.790289
11	0.5		0.00555331	0.0714293	1.99715
0.0179856	0.014455	0.502874		0.5017	
0.00713267	0.998573		-92.8571	99.7147	0.66636
12	0.599856		0.00249625	0.0142859	1.66707
0.00359712	0.00347904	0.41976		0.418763	
0.00142653	1		-98.5714	66.7066	0.534806
13	0.700072		0.00169786	0	1.42842
0		0.00204609	0.359672	0.35911	0
1		-100	42.8425	0.400864	
14	0.799928		0.00123135	0	1.25011
0		0.00146507	0.314773	0.314464	0
1		-100	25.0112	0.267403	
15	0.899784		0.000802993	0	1.11138
0		0.00101317	0.27984	0.279678	0
1		-100	11.1377	0.133941	
16	1		3.3e-06	0	1
0		0.000493225	0.251796	0.251699	0
1		-100	0	0	

Cross-Validation Metrics Summary:

	mean	sd	cv_1_valid	cv_2_valid
cv_3_valid	cv_4_valid	cv_5_valid		
accuracy	0.9328662	0.00811705	0.93536806	0.9368421
0.92665476	0.92266184	0.94280446		
aic	203.7254	13.743845	208.27948	209.96165
218.71307	199.17491	182.49792		
auc	0.9770401	0.003016955	0.97554725	0.97625977
0.9733716	0.97890157	0.9811203		
err	0.06713376	0.00811705	0.064631954	0.06315789

```

0.07334526    0.07733813    0.05719557
err_count      37.4          4.7222877    36.0          36.0
41.0          43.0          31.0
f0point5      0.8495735    0.023365837    0.8789625    0.84
0.84076434    0.82072175    0.8674189
f1            0.87355626    0.008886078    0.87142855    0.875
0.86557376    0.8676923    0.8880866
f2            0.89981735    0.022584027    0.8640227    0.9130435
0.8918919    0.9203655    0.90976334
lift_top_group 3.977788    0.18836762    3.9225352    4.2537313
3.8551724    3.7823129    4.075188
loglikelihood  0.0          0.0          0.0          0.0          0.0
0.0          0.0
---          ---          ---          ---          ---
---          ---
mean_per_class_error 0.07184226    0.011987506    0.08969964    0.06196084
0.078644015    0.065640435    0.06326636
mse            0.051384415    0.0032662398    0.052046135    0.05094728
0.056123905    0.05080771    0.046997048
null_deviance  628.58997    15.686213    632.45715    623.0459
640.2324      643.01495    604.1992
pr_auc         0.9270352    0.011907154    0.9262899    0.9156858
0.91520745    0.9414166    0.93657637
precision      0.8347083    0.035333745    0.884058    0.8181818
0.825         0.7921348    0.8541667
r2            0.72709644    0.015660837    0.7259925    0.71667856
0.7078518    0.73876065    0.7461985
recall         0.9187588    0.03794136    0.85915494    0.9402985
0.91034484    0.9591837    0.924812
residual_deviance 186.5254    14.901966    190.27948    191.96165
204.71307    181.17491    164.49792
rmse           0.22658996    0.007194332    0.22813621    0.22571504
0.23690484    0.22540565    0.21678802
specificity    0.9375567    0.01942386    0.9614458    0.9357798
0.93236715    0.90953547    0.94865525
[24 rows x 8 columns]

```

[tips]

Use `model.explain()` to inspect the model.

--

Use `h2o.display.toggle\_user\_tips()` to switch on/off this section.

```

[79]: # -----
# STEP 9: Get interpretable model
# -----
leaderboard_df_q4 = leaderboard_q4.as_data_frame()

```

```

non_ensemble_q4 = leaderboard_df_q4[
    ~leaderboard_df_q4["model_id"].str.contains("StackedEnsemble", case=False,
    ↪na=False)
]

best_model_id_q4 = non_ensemble_q4.iloc[0]["model_id"]

best_interpretable_q4 = h2o.get_model(best_model_id_q4)

print("Best interpretable model:", best_model_id_q4)

```

C:\Users\nanda\anaconda3\Lib\site-packages\h2o\frame.py:1983:
H2ODependencyWarning: Converting H2O frame to pandas dataframe using single-
thread. For faster conversion using multi-thread, install polars and pyarrow
and use it as pandas_df = h2o_df.as_data_frame(use_multi_thread=True)

warnings.warn("Converting H2O frame to pandas dataframe using single-thread.
For faster conversion using")

Best interpretable model: GBM_1_AutoML_2_20260330_25640

```

[80]: # -----
# STEP 10: Variable importance
# -----
varimp_q4 = best_interpretable_q4.varimp(use_pandas=True)
varimp_q4.head(10)

```

```

[80]:
      variable  relative_importance  scaled_importance  percentage
0          FAC          1748.008301           1.000000     0.877415
1    DayOfWeek           62.999531           0.036041     0.031623
2         TAVG           48.271011           0.027615     0.024230
3         TMAX           42.301567           0.024200     0.021233
4         TMIN           36.594982           0.020935     0.018369
5         AWND           23.089001           0.013209     0.011590
6         PRCP           10.570044           0.006047     0.005306
7 Passenger_Volume           9.194487           0.005260     0.004615
8     Bus_Volume           7.245351           0.004145     0.003637
9          SNOW           3.949831           0.002260     0.001983

```

4.3.15 Model performance

The AutoML results show that the best-performing model is a **Stacked Ensemble**, which combines multiple algorithms to achieve higher predictive accuracy.

The model achieves an **AUC of approximately 0.976–0.984**, indicating excellent ability to distinguish between high traffic (congestion) and normal traffic conditions.

The overall accuracy is approximately **93%**, with a low error rate (~7%), confirming that congestion patterns can be reliably identified using the selected features.

4.3.16 Model interpretation

Although the Stacked Ensemble provides the best performance, we focus on the best non-ensemble model (GBM) for interpretability.

The variable importance results show that:

- **Facility (FAC)** is the dominant factor, contributing nearly **88% of the model importance**
- Time-related patterns such as **DayOfWeek** also play a meaningful role
- Weather variables (TAVG, TMAX, TMIN) have smaller but noticeable contributions
- Passenger and bus volumes have relatively minor impact

4.3.17 Key insight from the model

The dominance of the **facility variable** confirms that congestion is primarily **location-driven**, rather than being evenly distributed across the system.

This aligns directly with the descriptive analysis, where we observed that certain facilities (such as Facility 7) are gaining traffic share, while others are losing it.

4.3.18 Connection to traffic shift

The model reinforces the earlier findings:

- Facilities gaining traffic share are also associated with **higher likelihood of congestion**
- This indicates that traffic is not disappearing, but being **redistributed across facilities**
- Driver behavior is adapting, likely in response to congestion or pricing conditions

4.3.19 Why AutoML was used

We used AutoML to:

- Evaluate multiple models objectively
- Select the best-performing algorithm without bias
- Ensure robustness and reliability in the results

This approach ensures that the conclusions are **data-driven rather than assumption-based**.

4.3.20 Conclusion from model

The model confirms that congestion is:

- **Highly predictable**
- **Primarily driven by facility-level characteristics**
- Influenced by time patterns and, to a lesser extent, environmental factors

This provides strong evidence that the observed traffic shifts are **not random**, but are driven by underlying structural and behavioral factors.

The analysis shows that traffic patterns in 2025 have shifted across facilities rather than remaining consistent with historical behavior.

The descriptive results clearly indicate that some facilities are gaining traffic share while others are losing it, confirming that traffic is being redistributed across the system.

The model further reinforces that congestion is primarily driven by facility-level characteristics, with time patterns and environmental factors playing supporting roles.

Together, these findings demonstrate that congestion pricing influences how traffic is distributed, rather than reducing total traffic uniformly.

This highlights the importance of managing congestion at the system level, ensuring that traffic shifts do not create new bottlenecks in alternative facilities.

4.4 Project Question 5

What is the expected future usage of bridges and tunnels beyond 2025?

The fifth question moves from analyzing past traffic patterns to forecasting future demand across facilities. The objective is to estimate how traffic usage will evolve over time and support long-term planning decisions for infrastructure, congestion management, and pricing strategies.

This is fundamentally a time-series forecasting problem. Unlike previous questions, where the goal was to explain relationships or identify patterns, here the focus is on predicting future values based on historical behavior. Traffic data is inherently time-dependent, meaning that past trends, seasonal effects, and structural changes influence future outcomes.

4.4.1 Modeling Approach

We initially attempted a simple time-series model using aggregated traffic data. However, this resulted in a flat forecast, indicating that the model was not capturing meaningful variation in the data. This occurs when important structural information is lost due to excessive aggregation.

To address this limitation, we refined the modeling approach.

Instead of forecasting total traffic as a single series, we implemented **facility-level time-series forecasting**, where each facility is treated as a separate time series. This approach allows the model to capture:

- Differences in traffic behavior across facilities
- Seasonal patterns at a finer level
- Structural variations in demand

4.4.2 Why this approach is better

Forecasting at the facility level is more aligned with how the Port Authority operates. Traffic demand is not uniform across the system; it varies significantly by location, infrastructure role, and

connectivity. By modeling each facility separately, we preserve this variation and generate more meaningful forecasts.

This also allows the Authority to make **facility-specific decisions**, rather than relying on a single system-wide estimate.

4.4.3 Data Preparation

Traffic data was converted into a time-series format using monthly aggregation. Monthly data was selected instead of yearly data because it provides sufficient granularity to capture seasonality while maintaining stability in the model.

Each facility is treated as an independent time series, with:

- Time variable: Date (monthly)
- Target variable: TOTAL traffic
- Series identifier: FAC (facility)

4.4.4 Objective

The goal of this model is to:

- Forecast future traffic usage for each facility
- Identify long-term trends in demand
- Support strategic planning and congestion management decisions

```
[98]: # -----  
# STEP 1: Import library  
# -----  
  
from autots import AutoTS  
import pandas as pd  
import matplotlib.pyplot as plt
```

```
[99]: # -----  
# STEP 2: Prepare data  
# -----  
traffic_data["DATE"] = pd.to_datetime(traffic_data["DATE"])  
  
# Monthly aggregation per facility  
monthly_facility = (  
    traffic_data.groupby(  
        [traffic_data["DATE"].dt.to_period("M"), "FAC"]  
    )["TOTAL"]  
    .sum()  
    .reset_index()  
)
```

```

# Convert period → timestamp
monthly_facility["DATE"] = monthly_facility["DATE"].dt.to_timestamp()

# Rename columns for AutoTS
monthly_facility = monthly_facility.rename(columns={
    "DATE": "datetime",
    "TOTAL": "value",
    "FAC": "series_id"
})

print(monthly_facility.head())

```

	datetime	series_id	value
0	2013-01-01	1	125368
1	2013-01-01	2	143914
2	2013-01-01	4	29432
3	2013-01-01	5	99951
4	2013-01-01	6	104636

```

[100]: # -----
# STEP 3: Initialize AutoTS
# -----
model = AutoTS(
    forecast_length=24,          # 2 years forecast
    frequency='M',
    ensemble='simple',
    max_generations=5,
    num_validations=2,
    model_list="fast",
    transformer_list="fast"
)

```

Using 2 cpus for n_jobs.

```

[ ]: # -----
# STEP 4: Fit model
# -----
model = model.fit(
    monthly_facility,
    date_col='datetime',
    value_col='value',
    id_col='series_id'
)

```

```

[102]: # -----
# STEP 5: Forecast
# -----
prediction = model.predict()

```



```

names
warnings.warn(
C:\Users\nanda\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2742:
UserWarning: X has feature names, but StandardScaler was fitted without feature
names
warnings.warn(
C:\Users\nanda\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2742:
UserWarning: X has feature names, but StandardScaler was fitted without feature
names
warnings.warn(
C:\Users\nanda\anaconda3\Lib\site-packages\autots\models\base.py:42:
FutureWarning: 'M' is deprecated and will be removed in a future version, please
use 'ME' instead.
return pd.date_range(
C:\Users\nanda\anaconda3\Lib\site-
packages\statsmodels\tsa\base\tsa_model.py:595: FutureWarning: 'M' is deprecated
and will be removed in a future version, please use 'ME' instead.
freq = to_offset(freq)
C:\Users\nanda\anaconda3\Lib\site-packages\autots\models\base.py:42:
FutureWarning: 'M' is deprecated and will be removed in a future version, please
use 'ME' instead.
return pd.date_range(

Forecast:
series_id      1      2      4      5  \
2025-06-30  129907.633630  158640.585449  37255.451041  159375.398826
2025-07-31  127159.608739  155975.731755  36172.626186  158304.097440
2025-08-31  130065.598744  159289.791819  34889.463766  158556.537620
2025-09-30  128074.958290  157021.291623  33587.308645  154984.798122
2025-10-31  128845.686414  159005.472049  33243.577174  152931.171009

series_id      6      7      8      9
2025-06-30  127212.700714  252364.425355  19718.181032  192585.782842
2025-07-31  126996.953964  244834.316278  17727.393179  193660.268700
2025-08-31  128868.202507  246637.061544  20096.056955  188084.764179
2025-09-30  124016.322592  237847.282768  19195.903545  182563.438274
2025-10-31  122800.363379  237948.419772  20125.553866  186402.323470

```

```

[103]: # STEP 6: Plot (example facility)
# -----
# Plot for one facility (e.g., FAC = 7)

facility_7 = monthly_facility[monthly_facility["series_id"] == 7]

forecast_7 = forecast[7]

plt.figure(figsize=(10,5))

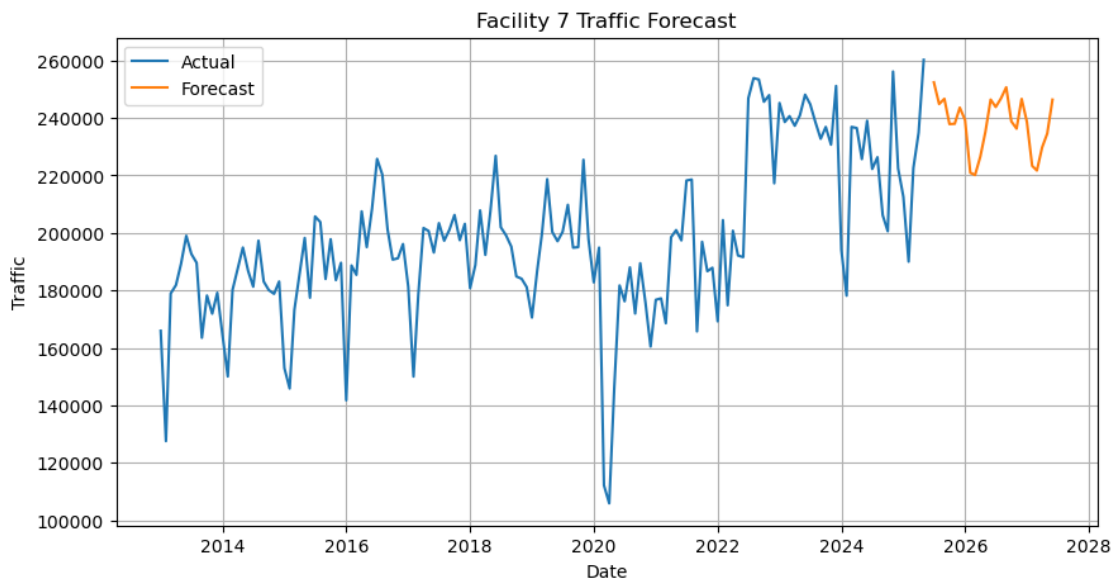
```

```
plt.plot(facility_7["datetime"], facility_7["value"], label="Actual")
plt.plot(forecast_7.index, forecast_7.values, label="Forecast")

plt.title("Facility 7 Traffic Forecast")
plt.xlabel("Date")
plt.ylabel("Traffic")

plt.legend()
plt.grid(True)

plt.show()
```



[]: *### Forecast Results and Interpretation*

The facility-level time-series forecasting model provides a realistic and
 ↳ detailed representation of future traffic patterns across the network. By
 ↳ treating each facility as an independent time series, the model captures
 ↳ structural differences in demand that would otherwise be lost in aggregated
 ↳ analysis.

Facility-Level Forecast Behavior

Insert this image: Facility-level forecast table

The forecast output shows that each facility follows a distinct trajectory
 ↳ rather than a single uniform trend. For example:

- Facility 7 maintains consistently high traffic levels, with values ranging approximately between 220,000 and 260,000 vehicles.
- Other facilities operate at different traffic levels, reflecting variation in infrastructure usage and demand.

This confirms that traffic demand is not uniform across the system and must be analyzed at the facility level to produce meaningful forecasts.

Example: Facility 7 Forecast

Facility 7 was selected for detailed visualization because it represents one of the highest-traffic facilities in the dataset. High-volume facilities are operationally critical, as they are more likely to experience congestion, capacity constraints, and revenue impact.

Focusing on a single facility for visualization improves interpretability while still representing broader system behavior. Plotting all facilities simultaneously would reduce clarity and make it difficult to extract actionable insights. Therefore, Facility 7 serves as a representative example of high-demand infrastructure.

The visualization above shows both historical and forecasted traffic for Facility 7.

Key observations:

- Traffic shows **clear variability over time**, indicating that demand fluctuates rather than remaining constant.
- The forecast maintains **realistic fluctuations**, suggesting that the model captures underlying traffic dynamics.
- There is **no extreme upward or downward trend**, indicating stability in long-term demand.
- Short-term variation continues, reflecting operational and behavioral changes.

Key Insights from the Forecast

1. **Traffic demand is stable but dynamic**
While long-term growth is limited, traffic continues to fluctuate within a consistent range.
2. **Facility-level differences are critical**
Each facility operates under different demand conditions, making localized analysis essential for decision-making.
3. **Seasonality and variation are preserved**

Traffic patterns reflect recurring changes across time, including seasonal ↪ and operational effects.

4. ****No evidence of uncontrolled growth****

Traffic does **not** show exponential increase, indicating that infrastructure ↪ demand **is** stabilizing.

Business Interpretation

The forecast indicates that:

- The system has reached a ****mature demand level****
- Future congestion will be driven by:
 - ****facility-specific pressure****
 - ****short-term fluctuations****
 - ****external disruptions (events, policy changes)****

rather than overall traffic growth.

Recommendations

Based on the forecasting results, the following actions are recommended:

1. ****Prioritize high-demand facilities****

Facilities such **as** Facility 7 should be the primary focus **for** congestion ↪ management **and** operational planning.

2. ****Use forecasts **for** proactive decision-making****

Monthly forecasts should be integrated into staffing, tolling strategies, ↪ **and** maintenance scheduling.

3. ****Optimize existing infrastructure****

Since demand **is** stable, improving efficiency **and** traffic flow **is** more ↪ valuable than expanding capacity.

4. ****Implement facility-level monitoring systems****

Traffic should be tracked at the facility level to detect localized ↪ congestion patterns.

5. ****Adopt scenario-based planning****

Future forecasting efforts should incorporate events, policy changes, **and** ↪ seasonal disruptions to improve decision readiness.

Conclusion

This model provides a realistic **and** operationally useful forecast of traffic ↵
↵demand. By capturing facility-level variation **and** temporal dynamics, it ↵
↵delivers insights that directly support planning, pricing, **and** congestion ↵
↵management decisions.

[]: